

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

Кафедра програмування

**КВАЛІФІКАЦІЙНА (БАКАЛАВРСЬКА)
РОБОТА**

СИМУЛЯТОР ЛІСОВИХ ПОЖЕЖ НА ОСНОВІ КЛІТИННИХ АВТОМАТІВ

Виконала: студентка групи ПМІ-46с
спеціальності 122 – комп'ютерні науки

(підпис) **Тригуб М.В.**
(прізвище та ініціали)

Керівник _____
(підпис) **Селіверстов Р.Г.**
(прізвище та ініціали)

Рецензент _____
(підпис) (прізвище та ініціали)

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СИМВОЛІВ, ОДИНИЦЬ, СКОРОЧЕНЬ І ТЕРМІНІВ.....	3
ВСТУП.....	4
1. ТЕОРЕТИЧНІ ОСНОВИ МОДЕЛЮВАННЯ ЛІСОВИХ ПОЖЕЖ ТА ВИБІР ПІДХОДУ.....	6
1.1. Лісові пожежі як об'єкт моделювання та основні фактори поширення.....	6
1.2. Огляд підходів та обґрунтування вибору клітинних автоматів.....	7
1.3. Модель Бак-Чен-Танга (ВСТ) як основа.....	9
1.4. Сусідство Мура та фон Неймана: порівняння.....	10
2. ПОСТАНОВКА ЗАДАЧІ ТА МАТЕМАТИЧНА МОДЕЛЬ СИМУЛЯТОРА.....	12
2.1. Постановка задачі.....	12
2.2. Математична модель клітинного автомата.....	14
2.2.1. Дискретний простір і час.....	14
2.2.2. Множина станів клітин.....	15
2.2.3. Сусідство клітинного автомата.....	16
2.2.4. Початкова генерація лісу.....	16
2.3. Моделювання факторів середовища.....	17
2.3.2. Займистість типів рослинності.....	19
2.3.3. Вплив стадій горіння.....	19
2.3.4. Вплив вітру та анізотропія поширення.....	20
2.4. Правила переходів станів.....	21
2.4.1. Займання від сусідніх клітин.....	21
2.4.2. Стохастичне займання від блискавок.....	22
2.4.3. Перехід між стадіями горіння.....	23
2.4.4. Вигорання, бар'єри та вплив дощу на згасання.....	24
2.5. Алгоритм кроку симуляції.....	24
2.6. Метрики оцінювання результатів.....	27
3. ПРОГРАМНА РЕАЛІЗАЦІЯ СИМУЛЯТОРА ЛІСОВИХ ПОЖЕЖ.....	30
3.1. Вибір технологічного стеку та обґрунтування.....	30
3.1.1. Python.....	30
3.1.2. Numpy.....	30
3.1.3. PySide6.....	30
3.1.4. Matplotlib.....	30
3.1.5. Pandas.....	30
3.1.6. Інструменти якості коду.....	30
3.2. Архітектура застосунку.....	30
3.2.1. Ядро моделі.....	30

3.2.2. Графічний інтерфейс.....	30
3.2.3. Серійні експерименти та аналітика.....	30
3.2.4. Точки входу та взаємодія між шарами.....	30
3.3. Реалізація ядра симулятора.....	30
3.4. Опис графічного інтерфейсу користувача.....	30
3.5. Інструменти інтерактивного редагування середовища.....	30
3.6. Серійні експерименти, збір метрик та експорт результатів.....	30
3.7. Тестування та відтворюваність результатів.....	30
4. РЕЗУЛЬТАТИ СИМУЛЯЦІЙНИХ ЕКСПЕРИМЕНТІВ.....	30
ВИСНОВКИ.....	31
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	31

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СИМВОЛІВ, ОДИНИЦЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

Анізотропія –

BCT (Bak-Chen-Tang model) – модель Бак-Чен-Танга для моделювання пожеж.

CA (Cellular automaton) – клітинний автомат (КА).

GUI (Graphical User Interface) – графічний інтерфейс користувача.

Qt / PySide6 – фреймворк Qt для Python та бібліотека PySide6 для створення GUI.

NumPy – бібліотека Python для роботи з багатовимірними масивами та чисельних обчислень.

Moore neighborhood (сусідство Мура) – околиця клітинки, що складається з 8 сусідів (по вертикалі, горизонталі та діагоналях).

Von Neumann neighborhood (сусідство фон Неймана) – околиця клітинки, що складається з 4 сусідів (по вертикалі та горизонталі).

Тороїдальна сітка –

Стохастичність – властивість систем або процесів, що характеризується випадковістю, недетермінованістю.

Пайплайн –

CLI –

Агрегація –

Трасованість –

OFAT –

BAF –

ВСТУП

Лісові пожежі в наш час є одними з найнебезпечніших природних явищ з якими стикається людина. Щорічно вони приносять великі екологічні, економічні та соціальні збитки, знищуючи цілі масиви лісів, спричиняючи деградацію ґрунтів та погіршення якості повітря [1]. Крім цього, вони завдають шкоди всьому біорізноманіттю, що населяє ту чи іншу територію і неодмінно шкодять життю та здоров'ю людей. Часто поширення пожеж є складно контрольованим, бо на їх динаміку впливають різні фактори, такі як: погодні умови, вітер, вологість, температура, тип рослинності, і, звісно ж, людський фактор. Тому станом на сьогодні, через погіршення клімату і збільшення кількості стихійних лих, прогнозування поведінки лісових пожеж стає надзвичайно важливим.

Сьогодні використовують різні підходи до моделювання лісових пожеж, я можу виділити фізичні, емпіричні та напівемпіричні. Кожен з цих підходів має як свої переваги так і обмеження. Наприклад якщо говорити про фізичні моделі, то вони надзвичайно точно описують процеси горіння, але при тому є ресурсо затратними і складними для обчислень. Коли ми розглянемо емпіричні та напівемпіричні моделі, то побачимо, що вони, навпаки, є швидшими, але при тому

можуть втрачати точність в нетипових або складних для себе умовах. Тому для практичного застосування, коли потрібно швидко та наочно побачити результат, мати змогу змінювати параметри та поекспериментувати, використовують імітаційні моделі – клітинні автомати.

Клітинним автоматом є така дискретна математична модель, яка складається з сітки поділеної на клітинки, стан кожної клітинки змінюється за певними правилами і залежить як від внутрішнього стану клітинки, так і від станів її сусідів. Саме підхід з клітинним автоматом добре підходить для опису таких явищ, які поширюються в просторі, зокрема пожеж. Використання клітинних автоматів має багато переваг, наприклад простота моделювання, невисокі обчислювальні витрати, а також легкість масштабування і наочність розвитку пожеж.

Актуальність даної теми полягає у потребі доступних середовищ для імітаційного моделювання лісових пожеж, які можна використовувати як для навчання так і для оцінки ризиків, прогнозування наслідків і демонстрації впливу різних факторів на розвиток і поширення вогню. Симулятор, що створений на основі клітинних автоматів дає можливість експериментувати з різними чинниками, що впливають на пожежі (вологість, температура, напрямок та сила вітру, тип рослинності, його щільність та займистість) та спостерігати за процесом в реальному часі і порівняти різні сценарії. Завдяки цьому ми можемо краще зрозуміти закономірності поширення лісових пожеж, сформувати уявлення про наслідки, що є корисним як для навчання так і для попереднього аналізу прикладних задач.

Метою цієї роботи є програмна розробка симулятора лісових пожеж на основі клітинних автоматів, який забезпечує моделювання і інтерактивну візуалізацію процесу займання та розповсюдження вогню з урахуванням обраних факторів, які можна змінювати для того, щоб відслідкувати різні сценарії і на їх основі побудувати статистику.

1. ТЕОРЕТИЧНІ ОСНОВИ МОДЕЛЮВАННЯ ЛІСОВИХ ПОЖЕЖ ТА ВИБІР ПІДХОДУ

1.1. Лісові пожежі як об'єкт моделювання та основні фактори поширення

Лісова пожежа є складним процесом, під час якого швидко змінюється стан території через взаємодію багатьох чинників. Загалом пожежу можна розділити на 3 стадії: перша – займання, друга – розвиток (фронт горіння поширюється) і останньою стадією є згасання. Для практичних задач оцінювання динаміки є одним з ключових факторів, бо важливо враховувати швидкість та напрям розповсюдження, площу, яка зазнає ураження і, звісно ж, вплив зовнішнього середовища. Тому лісові пожежі є актуальною проблемою для створення комп'ютерних та математичних моделей, що дозволяє досліджувати різні сценарії розгортання подій та оцінювати кореляцію між силою пожежі та факторами, що її спричиняють.

Поширення пожежі визначається наявністю “палива” (рослини, дерева), бо без нього пожежа не мала б чим “живитися”. Також важливими є умови навколишнього середовища і щільність та займистість рослинного покриву. У реальних умовах займистість залежить від вологості, порід дерев, сезонності,

погодних умов і ще величезна кількість факторів. Саме це робить повноцінні фізичні моделі “важкими” для обчислень, тому для своєї імітаційної моделі я виділила ключові параметри, які найбільше впливають на результат.

Найзначущі фактори, що впливають на виникнення і швидкість та напрям поширення пожежі:

- **Вітер** та його сила визначає куди буде розповсюджуватися пожежа і з якою швидкістю це буде відбуватися. Часто пориви вітру спричиняють перенесення тліючих частинок чи іскор у напрямку його руху, що може спричинити займання ділянок попереду основної пожежі. Цей процес називається анізотропією (в напрямку вітру ймовірність загоряння збільшується, а проти вітру – зменшується).
- **Вологість** в свою чергу напряму впливає на легкість займання та інтенсивність горіння. Коли вологість підвищена, частина енергії витрачається на випаровування води – займання важче та повільніше, в сухих умовах навпаки, поширення пожежі стає швидшим та стабільнішим.
- **Температура** є також важливим чинником, бо вона впливає на вологість та стан палива, при більшій температурі висушується рослинність, що збільшує ризик займання. Чим нижча температура, тим менший ризик займання.
- **Тип рослинності та її займистість** є вагомим фактором, оскільки рослинність виступає “паливом” для горіння, різні типи рослинності можуть мати різну займистість. Наприклад, хвойні культури зазвичай займаються легше ніж листяні через більший вміст смол.
- **Перешкоди** (ділянки, що не підтримують горіння) також впливають на траєкторію поширення і масштаби пожежі. Їх наявність допомагає зупинити або змінити напрям поширення вогню.
- **Джерело займання** може бути як природним (наприклад, блискавка), так і антропогенним (підпал, необережне поводження з вогнем).

1.2. Огляд підходів та обґрунтування вибору клітинних автоматів

Моделювання лісових пожеж може виконуватись різними підходами, вони мають різну обчислювальну складність і точність, але загалом можна виділити 3 категорії: фізичні, емпіричні та імітаційні.

Фізично-орієнтовані моделі стараються найбільш точно і детально описати процеси горіння і взаємодії полум'я з навколишнім середовищем, але при тому вони і вимагають якомога більше точних вхідних даних. Крім того ці моделі витрачають багато ресурсів для обчислень, тому часто для задач, де важливі експерименти з можливістю швидкого коригування чинників, вони є заскладними.

Емпіричні та напівемпіричні моделі базуються на попередніх експериментах і типових закономірностях. Вони дозволяють досить швидко оцінювати пожежі, але оскільки вони пристосовані до певних умов і чинників, то можуть бути менш точними при їх зміні.

Імітаційні моделі описують пожежі не через детальну фізику процесу, а як сукупність локальних взаємодій. До такого підходу відносять дискретні просторові, агентні та гібридні моделі. Їхня перевага в тому, що вони дозволяють швидко змінювати правила і параметри, перевіряти та порівнювати різні сценарії. Саме тому клітинні автомати є частим інструментом для моделювання пожеж.

Клітинний автомат представляють як територію на дискретній сітці, де кожна клітинка має стан (наприклад, “порожньо”, “горить”, “дерево”) і за певними правилами переходів, що залежить від поточного стану клітинки і стану її 8-ми (в випадку який я розглядаю) сусідів, може змінювати цей стан на кожному кроці часу. Така модель добре підходить для моделювання пожеж, бо загорання зазвичай відбувається локально – від сусідніх клітин або від раптової події (наприклад, блискавка).

Ось декілька фактів якими я керувалась, коли обирала клітинні автомати для даної роботи:

- Пожежа поширюється з просторовим характером, отже подання території у вигляді двовимірної сітки, а часу як дискретних кроків є логічним та підходящим способом для того, щоб коректно описати динаміку.
- Правила переходів між станами клітинного автомата враховують і різні фактори середовища (вологість, температура і т.д.) і різні типи рослинності та наявність перешкод.
- Оновлення стану кожної клітинки залежить лише від її локального оточення, тому модель має невисокі обчислювальні витрати і її можна масштабувати на різні розміри сіток без ускладнення самого алгоритму.
- Стан сітки легко відобразити графічно, а також реалізувати різні інструменти редагування (наприклад підпал, посадка дерев різного типу, розставляння бар'єрів, стирання) і покроковий режим. Окремою перевагою є те, що вона є інтуїтивно зрозуміла.
- Можна швидко змінювати початкові умови та різні параметри (наприклад, щільність засадження, вологість і т.д.), що дає простір для експериментів та порівняння результатів моделювання. Наприклад, можна оцінити тривалість пожежі, площу вигорання та інші фактори за різних умов.

1.3. Модель Бак-Чен-Танга (BCT) як основа

Модель Бак-Чен-Танга (Bac-Chen-Tang, BCT) є однією з класичних моделей для опису процесів на кшталт лісових пожеж. Вона належить до класу клітинно-автоматних моделей і була створена для того, щоб спростити дослідження поширення пожеж на дискретній сітці. Ця модель зображується як решітка клітин, де кожна клітина може перебувати в одному з кількох станів, а перехід між станами відбувається покроково відповідно до зазначених локальних правил. Модель BCT також розглядають в дослідженнях самоорганізованої критичності, а саме поведінки систем, в яких незначні локальні події можуть призводити до масштабних наслідків без зовнішнього точного налаштування параметрів.

У початковому варіанті модель подає ліс у вигляді двовимірної сітки, де кожна клітинка може перебувати в одному із 3 станів: порожня, дерево або дерево, що горить. Динаміка системи відповідно залежить від росту дерев, виникнення пожежі та вигорання палаючих клітин. На кожному кроці часу порожня клітина може стати деревом з певною ймовірністю p , дерево може загорітись через випадкову подію f або ж через палаючого сусіда, а клітинка в стані горіння після одного кроку переходить у порожній стан.

Отже, модель ВСТ відображає циклічний процес: порожня клітинка $\rightarrow$ дерево $\rightarrow$ горіння $\rightarrow$ порожня клітинка. Незважаючи на цю простоту моделі, вона дозволяє показувати пожежі з різною динамікою поширення: можуть утворюватись ділянки різної щільності лісу, а локальне займання може стати причиною пожежі різного масштабу зважаючи на поточний стан сітки.

Класична ВСТ-модель є дуже спрощеною, вона не враховує тип рослинності, її займистість, вологість, температуру, силу та напрям вітру, а також бар'єри. До того ж, загорання від сусідньої клітинки зазвичай розглядається як детермінований процес, тобто дерево займається завжди, коли біля нього є палаючий сусід. В реальності поширення пожежі залежить від набагато більшої кількості факторів, тому для симулятора цю базову модель варто розширити.

Таким чином, ВСТ-модель в цій роботі відіграє роль базової клітинно-автоматної схеми, на яку накладаються додаткові правила та параметри: вологість, температура, вітер, дощ, різні типи рослинності та бар'єри. Це дозволяє зберегти простоту класичного підходу, але водночас розширити його для моделювання впливу реальних факторів на середовище.

1.4. Сусідство Мура та фон Неймана: порівняння

Вибір сусідства відіграє важливу роль для моделей на основі клітинних автоматів, тому що саме воно визначає, які клітини будуть впливати на зміну стану поточної клітинки. Тому для моделювання лісової пожежі це також є важливим, бо вогонь поширюється не миттєво по всій території, а поступово: клітинки з рослинністю можуть загорітися від сусідніх палаючих ділянок.

Для двовимірних клітинних автоматів найчастіше застосовують два типи сусідств: сусідство Мура і сусідство фон Неймана. Вони відрізняються лише тим, яка кількість клітин буде задіяна для обрахунку стану поточної клітини на наступному кроці.

Отже, спершу розглянемо сусідство фон Неймана (рис. 1.4.1). Воно задіює чотири найближчі клітини, які мають спільну сторону з поточною: зверху, знизу, ліворуч та праворуч. Для клітини з координатами (i, j) формула для подання такого сусідства буде виглядати так:

$$N_v(i, j) = \{(i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)\}, \quad (1.1)$$

де

- $N_v(i, j)$ – множина сусідів клітини (i, j) у сусідстві фон Неймана.
- i – номер рядка клітини у сітці.
- j – номер стовпця клітини у сітці.
- $(i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)$ – верхній, нижній, лівий та правий сусіди поточної клітини відповідно.

Цей тип сусідства є досить простим для реалізації і потребує меншої кількості перевірок на кожному кроці симуляції. Проте оскільки вогонь тут може поширюватись лише в чотирьох напрямках: по горизонталі та по вертикалі, то це може спотворювати фронт поширення пожежі. Таким чином форма пожежі виглядатиме дуже штучно і буде менш наближена до реального поширення пожежі в просторі.

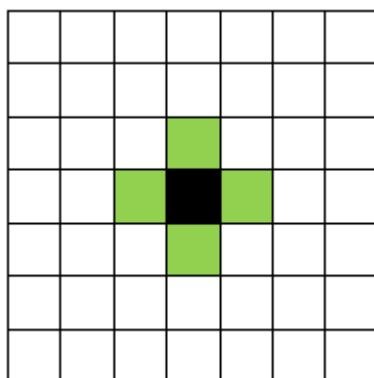


Рис. 1.4.1 Сусідство фон Неймана

Перейдемо до сусідства Мура (рис. 1.4.2), воно є розширенням сусідства фон Неймана. На відміну від останнього сусідство Мура враховує вже 8 клітин навколо поточної: чотири клітини по горизонталі та вертикалі і чотири по діагоналі. Для клітини (i, j) формула для подання сусідства Мура виглядатиме так:

$$N_M(i, j) = \{(i + a, j + b) \mid a, b \in \{-1, 0, 1\}, (a, b) \neq (0, 0)\}, \quad (1.2)$$

де

- $N_M(i, j)$ – множина сусідів клітини (i, j) у сусідстві Мура.
- i – номер рядка клітини у сітці.
- j – номер стовпця клітини у сітці.
- a та b – зміщення відносно поточної клітини по рядку та стовпцю.

Отже, в сусідстві Мура поточна клітина залежить від всіх найближчих клітин, які її оточують. Такий принцип краще показує просторовий характер поширення процесів на площині, бо вплив може передаватись по більшій кількості осей.

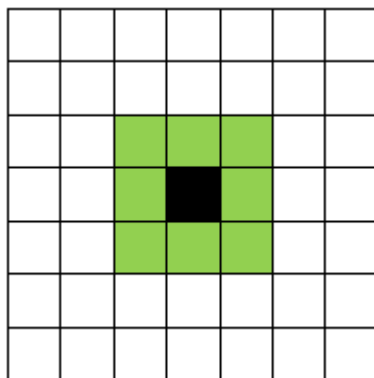


Рис. 1.4.2 Сусідство Мура

Від вибору сусідства залежить характер поширення фронту пожежі в моделі. Сусідство фон Неймана є простішим і не вимагає багато перевірок, але при тому задає лише чотири напрямки для взаємодії. Сусідство Мура в свою чергу потребує більше перевірок, але також дозволяє моделювати поширення пожежі аж у восьми напрямках, що дозволяє отримувати гнучкішу форму фронту пожежі. Також важливою перевагою сусідства Мура є його відповідність моделям, де потрібно

враховувати напрямок вітру, оскільки воно охоплює вісім можливих напрямків поширення.

2. ПОСТАНОВКА ЗАДАЧІ ТА МАТЕМАТИЧНА МОДЕЛЬ СИМУЛЯТОРА

2.1. Постановка задачі

У цій дипломній роботі об'єктом дослідження є процес поширення лісової пожежі у дискретному просторовому середовищі. Предметом дослідження є модель поширення пожежі на основі клітинного автомата, яка враховує такі параметри, як типи рослинності, вологість, температура, вітер, дощ, бар'єри та випадкові джерела займання.

Метою цієї роботи є розробка програмного симулятора поширення лісової пожежі на основі клітинного автомата, що зможе моделювати та інтерактивно візуалізувати процес займання та поширення вогню. Це також дозволить проводити сценарні експерименти та обчислювати кількісні метрики наслідків пожежі.

Для досягнення поставленої мети потрібно виконати наступні завдання:

1. створити дискретну модель лісового середовища у вигляді двовимірної сітки клітин з визначеною множиною станів.
2. реалізувати стохастичні правила переходів між станами клітин, враховуючи сусідні палаючі клітини.
3. врахувати вплив погодних умов, включаючи вологість, температуру, вітер, дощ і блискавки, на ймовірність займання.
4. забезпечити підтримку різних типів рослинності з індивідуальними коефіцієнтами займистості.
5. реалізувати інтерактивний графічний інтерфейс користувача з можливістю ручного редагування середовища та керування симуляцією.
6. реалізувати серійні експерименти з підтримкою різних сценаріїв та багатьох випадкових прогонів.
7. обчислювати кількісні метрики наслідків пожежі для порівняння результатів.

Вхідними параметрами моделі є розміри сітки, початкова щільність лісу, частка хвойних дерев, коефіцієнти займистості хвойних та листяних дерев, вологість, температура, напрям і сила вітру, інтенсивність дощу, параметри для блискавки та зерно генератора випадкових чисел. Додатково початкові умови можуть задаватись вручну через інтерфейс користувача, зокрема шляхом розміщення осередків займання, дерев або бар'єрів.

Вихідними даними моделі є кінцевий стан сітки та часовий ряд кількості палаючих клітин. Крім того, є набір метрик таких, як частка вигорілої площі, максимальна кількість одночасно палаючих клітин, тривалість пожежі, час до піку горіння, час до згасання і просторові характеристики вигорілої зони.

Дана модель є імітаційною, тобто вона не претендує на точне фізичне відтворення пожежі на конкретній місцевості. В межах роботи не використовуються реальні географічні дані, рельєф, детальні фізичні характеристики палива та швидкість вітру у фізичних одиницях. Основне призначення моделі полягає у порівнянні сценаріїв та дослідженні впливу параметрів середовища на динаміку і масштаби пожежі.

2.2. Математична модель клітинного автомата

Математична модель симулятора побудована на основі класичної моделі Бак-Чен-Танга, розширеної за допомогою додавання двох типів рослинності з відповідними коефіцієнтами займистості, трьох стадій горіння, а також з урахуванням вологості, температури, вітру, дощу та випадкових або ручних займань. Двовимірною прямокутною сіткою клітин слугує моделі лісовим середовищем. Кожна клітина сітки може перебувати в одному з дискретних станів у кожен момент часу. Час тут також є дискретним, тобто симуляція відбувається покроково, а перехід від стану G^t до G^{t+1} відбувається відповідно до правил переходу, які враховують як поточний стан клітини, так і стани її безпосередніх сусідів та параметри середовища.

Формально клітинний автомат можна подати в такому вигляді:

$$CA = (L, S, N, F, \Theta), \quad (2.1)$$

де L – множина клітин сітки;
 S – множина станів клітини;
 N – функція сусідства;
 F – функція переходу станів;
 Θ – множина параметрів моделі.

2.2.1. Дискретний простір і час

Простір моделі задається двовимірною прямокутною сіткою, розміром $H \times W$, де H – це висота, а W – ширина поля. Кожна клітина сітки має координати (i, j) , де $i = 0, \dots, H - 1$, а $j = 0, \dots, W - 1$. Звідси множина всіх клітин буде мати такий вигляд:

$$L = \{(i, j) \mid i = 0, 1, \dots, H - 1; j = 0, 1, \dots, W - 1\} \quad (2.2)$$

Стан клітини (i, j) в момент часу t позначається як $s_{i,j}^t$, тоді стан всієї сітки можна подати так:

$$G^t = \{s_{i,j}^t \mid (i, j) \in L\}, \quad (2.3)$$

де G^t – конфігурація всієї сітки в момент часу t .

У моделі поняття часу є дискретним, тобто $t = 0, 1, 2, \dots, T$, де T – максимальна кількість кроків симуляції. На кожному кроці симуляції стани усіх клітинок синхронно оновлюються, тобто на основі конфігурації сітки G^t формується нова конфігурація G^{t+1} . Один крок є умовною дискретною одиницею часу, тобто він не прив'язується до конкретної фізичної одиниці.

2.2.2. Множина станів клітин

Кожна клітина сітки даної моделі в кожен момент часу перебуває в одному стані з цієї скінченної множини:

$$S = \{E, D, C, B_1, B_2, B_3, R, X\}, \quad (2.4)$$

де E – порожня клітина;
 D – листяне дерево;
 C – хвойне дерево;

B_1, B_2, B_3 – перша, друга та третя стадії горіння відповідно;

R – бар'єр;

X – вигоріла клітинка.

В моделі розрізняються два типи дерев для того, щоб враховувати неоднорідність лісового середовища. Крім того листяні та хвойні типи рослинності можуть мати різну схильність до займистості, тому в моделі задаються окремі коефіцієнти для кожного типу.

Для горіння виділяється три окремі стадії, що дає можливість відобразити поступове згасання, а не миттєве вигорання клітини. На початковій стадії горіння клітина має більший вплив на займання сусідів, ніж клітина на пізнішій стадії.

2.2.3. Сусідство клітинного автомата

Для визначення взаємодії між клітинами використовується сусідство Мура. Тобто на поточний стан клітини впливає вісім її сусідів: чотири ортогональні та чотири діагональні клітини. Для клітини (i, j) сусідство можна подати таким чином:

$$N(i, j) = \{(i + \Delta i, j + \Delta j) \mid \Delta i, \Delta j \in \{-1, 0, 1\}, (\Delta i, \Delta j) \neq (0, 0)\}, \quad (2.5)$$

де

$N(i, j)$ – множина сусідніх клітин для клітини (i, j) ;

Δi – зміщення по рядку;

Δj – зміщення по стовпцю;

$(\Delta i, \Delta j) \neq (0, 0)$ – умова, яка показує, що поточна клітина не входить до множини своїх сусідів.

Як було зазначено в підрозділі 1.4, сусідство Мура обрано з наступних причин. По-перше, воно дозволяє моделювати поширення пожежі одразу в восьми напрямках, що показує природніший формат фронту горіння порівняно з сусідством фон Неймана, де задіяно лише чотири ортогональні сусіди. По-друге, для того, щоб коректно показати анізотропію вітру принципово охоплювати саме вісім напрямків, оскільки основна класифікація вітру враховує сам їх. Клітини, що розташовані на межі сітки враховують лише тих сусідів, які належать області

моделювання. Тобто циклічний перехід через межі сітки не використовується, бо сітка не є тороїдальною.

2.2.4. Початкова генерація лісу

Початкова конфігурація лісу формується стохастично, тобто випадково. Спочатку всі клітини вважаються порожніми. Потім незалежно для кожної клітини визначається її початковий стан. Це відбувається за допомогою параметрів густоти лісу ρ та частки хвойних дерев q .

З ймовірністю ρ в клітині з'являється дерево, а з ймовірністю $1 - \rho$ вона залишається порожньою. Якщо дерево створене, то його тип визначається параметром q . Дерево буде хвойним з ймовірністю q , а з ймовірністю $1 - q$ листяним.

Отже, повні ймовірності початкового стану кожної клітини (i, j) виглядатимуть так:

$$P(s_{i,j}^0 = E) = 1 - \rho, \quad (2.6)$$

$$P(s_{i,j}^0 = C) = \rho q, \quad (2.7)$$

$$P(s_{i,j}^0 = D) = \rho (1 - q), \quad (2.8)$$

де P – ймовірність відповідного початкового стану клітини;

$s_{i,j}^0$ – стан клітини (i, j) на початку симуляції;

E, C, D – порожня клітина, хвойне дерево, листяне дерево відповідно;

$\rho \in [0, 1]$ – густина лісу;

$q \in [0, 1]$ – частка хвойних дерев серед усіх дерев лісу.

Для того, щоб мати можливість відтворювати експерименти у моделі використовується зерно генератора випадкових чисел. Це дозволяє отримувати однакову початкову конфігурацію при однакових параметрах

2.3. Моделювання факторів середовища

У даній моделі зовнішні умови середовища не змінюють структуру клітинного автомата, але вони впливають на ймовірність займання дерева від сусідньої палаючої клітини. Такими факторами є вологість, температура, дощ, тип рослинності, стадія горіння клітини та вітер. Усі вони зводяться до єдиної формули ефективної ймовірності займання, яку можна подати в такому вигляді:

$$P(\Delta i, \Delta j) = \Pi_{[0,1]} \left(W(\Delta i, \Delta j) \cdot D_{eff} \cdot F_{veg} \cdot F_{stage} \right), \quad (2.9)$$

де $\Pi_{[0,1]}(x)$ – оператор обмеження значення x відрізком $[0, 1]$;

$W(\Delta i, \Delta j)$ – вітровий множник для напрямку поширення $(\Delta i, \Delta j)$;

D_{eff} – ефективна сухість середовища;

F_{veg} – коефіцієнт займистості типу рослинності;

F_{stage} – множник стадії горіння сусідньої палаючої клітини розташованої в напрямку $(\Delta i, \Delta j)$.

Кожен з цих множників описується детальніше в наступних підпунктах.

2.3.1. Ефективна сухість (вологість, температура, дощ)

Для того, щоб коректно враховувати погодні умови в моделі вводиться показник ефективної сухості D_{eff} . Він об'єднує вплив на середовище вологості, температури та дощу.

Вологість $H \in [0, 1]$ задається як вхідний параметр моделі. Базова сухість середовища визначається такою формулою:

$$D_h = 1 - H, \quad (2.10)$$

В даній моделі температура T нормується на інтервалі від $T_{min} = -10^\circ\text{C}$ до $T_{max} = 40^\circ\text{C}$ (цей інтервал можна змінити) за наступною формулою:

$$T_{norm} = \Pi_{[0,1]} \left((T - T_{min}) / (T_{max} - T_{min}) \right) \quad (2.11)$$

На основі нормованої температури формується температурний множник, в моделі він задається таким чином:

$$F_T = 0,5 + T_{norm} \quad (2.12)$$

Додавання сталої 0,5 потрібно для того, щоб навіть за мінімального значення T_{norm} вплив температури був ненульовим. Таким чином, значення F_T набуває значень в діапазоні $[0,5; 1,5]$: вища температура збільшує показник, нижча – зменшує.

Поточна інтенсивність дощу R_t , визначається як сума ручного та сценарного дощу:

$$R_t = \Pi_{[0,1]}(R_{man} + R_{sc}(t)), \quad (2.13)$$

де R_{man} – постійна інтенсивність дощу, що задається вручну;

$R_{sc}(t)$ – сценарний дощ, який активується лише в заданому часовому інтервалі.

Сценарний дощ задається інтенсивністю та інтервалом дії:

$$t_{start} \leq t < t_{end}, \quad (2.14)$$

де t_{start} , t_{end} – початковий та кінцевий кроки дії сценарного дощу відповідно.

Отже, звідси формула ефективної сухості виглядатиме так:

$$D_{eff} = \Pi_{[0,1]}(D_h \cdot F_T \cdot (1 - R_t)), \quad (2.15)$$

де D_h – базова сухість;

F_T – температурний множник;

R_t – поточна інтенсивність дощу.

Таким чином, підвищення вологості або інтенсивності дощу зменшує D_{eff} , тоді як підвищення температури навпаки збільшує. Показники температури та вологості не впливають одне на одного напряму, вони задаються як незалежні вхідні параметри і їхній спільний вплив враховується через показник ефективної сухості D_{eff} .

2.3.2. Займистість типів рослинності

Модель також враховує тип рослинності клітини. Коефіцієнт займистості $F_{veg}(i, j)$ для листяного дерева ($s_{ij} = D$) дорівнює f_D , для хвойного дерева ($s_{ij} = C$) відповідно f_C , а для всіх інших станів клітин дорівнює нулю.

За замовчуванням хвойні дерева мають вищий коефіцієнт займистості, ніж листяні. Це відповідає реаліям пожежної небезпеки хвойних рослин, оскільки вищий вміст смол підвищує ризик загоряння.

2.3.3. Вплив стадій горіння

Кожна стадія горіння B_1, B_2, B_3 має відповідний коефіцієнт на займання сусідніх дерев: $\alpha_1, \alpha_2, \alpha_3$. За замовчуванням ці коефіцієнти дорівнюють 0.8, 0.55 та 0.25 відповідно, тобто на першій стадії горіння клітина має найбільший вплив на сусідів, а на третій найменший. Отже, вплив палаючої клітини для певного напрямку поширення можна записати так:

$$F_{stage} = \alpha_1 I(B_1) + \alpha_2 I(B_2) + \alpha_3 I(B_3), \quad (2.16)$$

де $\alpha_1, \alpha_2, \alpha_3$ – коефіцієнти впливу відповідних стадій горіння;

$I(B_k)$ – індикатор, який показує чи перебуває сусідня клітина в стадії горіння B_k :

$I(B_k) = 1$, якщо клітина горить і $I(B_k) = 0$, якщо ні.

2.3.4. Вплив вітру та анізотропія поширення

Вітер задає анізотропію поширення пожежі, тобто залежність ймовірності займання клітини від напрямку вітру. Якщо вітер вимкнений або його сила дорівнює нулю, то $W = 1$ для всіх напрямків, тобто вітер ніяк не впливає на ймовірність займання.

Щоб враховувати напрям вітру необхідно обчислити косинус кута між напрямом поширення $(\Delta i, \Delta j)$ та вектором вітру (w_i, w_j) , ось як виглядатиме формула:

$$\gamma(\Delta i, \Delta j) = (\Delta i \cdot w_i + \Delta j \cdot w_j) / \left(\sqrt{\Delta i^2 + \Delta j^2} \cdot \sqrt{w_i^2 + w_j^2} \right) \quad (2.17)$$

Сила вітру $s \in [0, 1]$ використовується для визначення наступних базових множників:

$$W_{down} = 1 + s, \quad (2.18)$$

$$W_{cross} = 1 - 0,25s, \quad (2.19)$$

$$W_{up} = 1 - s, \quad (2.20)$$

де W_{down} – множник поширення за вітром;

W_{cross} – множник бокового поширення;

W_{up} – множник поширення проти вітру.

Для проміжних напрямків напрямків вітровий множник визначається через лінійну інтерполяцію. Якщо $\gamma(\Delta i, \Delta j) \geq 0$, тобто напрям поширення йде за вітром, то формула вітрового множника для напрямку $(\Delta i, \Delta j)$ виглядатиме так:

$$W(\Delta i, \Delta j) = W_{cross} + \gamma(\Delta i, \Delta j) \cdot (W_{down} - W_{cross}) \quad (2.21)$$

А якщо $\gamma(\Delta i, \Delta j) < 0$, тобто напрям поширення має складову проти вітру, то формула вітрового множника для напрямку $(\Delta i, \Delta j)$ матиме такий вигляд:

$$W(\Delta i, \Delta j) = W_{cross} + (-\gamma(\Delta i, \Delta j)) \cdot (W_{up} - W_{cross}) \quad (2.22)$$

Отже, поширення за вітром підсилюється, поширення в бокових напрямках помірно послаблюється, а поширення проти вітру зазнає найбільшого штрафу. Таким чином, вітер не змінює рівномірно загальну інтенсивність горіння, а тільки перерозподіляє ймовірність поширення між різними напрямками.

2.4. Правила переходів станів

Клітинний автомат на кожному кроці часу синхронно оновлює стан всіх клітин сітки. Тобто спочатку розглядається поточна конфігурація сітки G_t , після цього на її основі формується нова сітка G_{t+1} . Зміна стану клітини на новий залежить від багатьох факторів таких, як її поточний стан, стани сусідніх клітин, параметри погодних умов, тип рослинності та випадкових подій в процесі симуляції. Основними переходами моделі є займання дерева від сусідньої

палаючої клітини або від блискавки, переходи між стадіями горіння, вигорання клітини, а також збереження станів згорілих клітин та бар'єрів.

2.4.1. Займання від сусідніх клітин

Займання поточної клітини від сусідніх клітин проєктується як стохастичний процес. При тому зайнятість можуть лише клітини, які перебувають в стані листяного або хвойного дерева, тобто $s_{ij}^t \in \{D, C\}$.

На кожному кроці для кожного напрямку сусідства $(\Delta i, \Delta j) \in N(i, j)$, визначеного формулою (2.5), перевіряється наявність палаючої клітини. Коефіцієнт F_{stage} відповідно до формули (2.16) визначає вплив сусідньої палаючої клітини на поточну. Якщо $F_{stage} > 0$ і поточна клітина перебуває в стані дерева, то ефективна ймовірність займання обчислюється за формулою (2.9).

Оскільки займання є стохастичним, то для кожного напрямку генерується така випадкова величина $u \sim U(0, 1)$, а займання в цьому напрямі відбувається, якщо виконується наступна умова:

$$u < P(\Delta i, \Delta j) \quad (2.23)$$

Через те, що модель перевіряє всі вісім напрямків сусідства, то результат займання по різних напрямках об'єднується. Клітина вважається такою, що загоряється, якщо умова (2.23) виконалась хоча б для одного з перевірюваних напрямків.

2.4.2. Стохастичне займання від блискавок

Крім того, що горіння може поширюватись від сусідніх клітин, в моделі враховується також зовнішнє випадкове джерело займання, тобто блискавка. Базова ймовірність події блискавки f зменшується за наявності дощу, це відповідає реальним умовам у природі, оскільки дощ є природним наслідком того, що висхідні потоки повітря слабшають, а електричні заряди всередині хмари починають розряджатись та нейтралізуватись. Формула за якою це відбувається у моделі виглядає таким чином:

$$P_f = \Pi_{[0,1]} \left(f \cdot (1 - R_t)^2 \right), \quad (2.24)$$

де R_t – поточна інтенсивність дощу, визначена формулою (2.13).

Якщо подія блискавки вимкнена або модель перебуває в активному періоді очікування між подіями, то нова подія не створюється. Після успішної події встановлюється певний час відновлення, який триває C кроків (ця кількість кроків задається користувачем і може змінюватись протягом симуляції). Якщо подія відбулась, то модель випадково обирає від 1 до K_{max} клітин які перебувають в стані дерева, це відбувається за наступною формулою:

$$K_{max} = \min(K, N_{tree}), \quad (2.25)$$

де K – максимальна кількість ударів за одну подію, що вказується в моделі;

N_{tree} – кількість доступних клітин, що перебувають в стані дерева.

Вибір клітини для займання від блискавки є зваженим, тобто не всі клітини з деревом мають однакову ймовірність бути вибраними. Для кожної клітини з деревом вводиться певний показник сприйнятливості до займання S_{ij} . Він показує, наскільки клітина (i, j) є придатною до займання при поточних умовах середовища. Сприйнятливості клітини можна визначити таким чином:

$$S_{ij} = \Pi_{[0,1]} \left(D_{eff} \cdot F_{veg}(i, j) \right) \quad (2.26)$$

Отже, якщо середовище сухе, а дерево має високий коефіцієнт займистості, то така клітина отримає більше значення S_{ij} і, відповідно, більшу ймовірність бути вибраною для удару блискавки. А якщо середовище вологе або клітина не є у стані дерева, то її сприйнятливості відповідно буде низькою або нульовою.

Ймовірність вибору конкретної клітини (i, j) можна визначити нормуванням її сприйнятливості відносно сприйнятливості всіх клітин, що містять дерево, ось як виглядає відповідна формула:

$$p_{ij} = S_{ij} / \sum_{(i,j)} S_{ij} \quad (2.27)$$

Дерева, які вибрані таким чином переходять до першої стадії горіння B_1 .

2.4.3. Перехід між стадіями горіння

Отже, дерево після займання переходить до першої стадії горіння B_1 . Після цього ця палаюча клітина послідовно проходить наступні дві стадії B_2 та B_3 і далі стає згорілою клітиною X . Ось як це виглядає схематично:

$$B_1 \rightarrow B_1 \rightarrow B_1 \rightarrow X \quad (2.28)$$

Якщо додатковий вплив дощу відсутній, то перехід між стадіями горіння відбувається на кожному наступному кроці часу. Для перших двох стадій це можна записати так:

$$s_{ij}^t = B_k \Rightarrow s_{ij}^{t+1} = B_{k+1}, \quad k = 1, 2 \quad (2.29)$$

Для третьої стадії перехід задається окремо:

$$s_{ij}^t = B_3 \Rightarrow s_{ij}^{t+1} = X \quad (2.30)$$

Таким чином, наявність цих кількох стадій горіння дає нам змогу показати поступове зниження його інтенсивності. Це також узгоджується з коефіцієнтами $\alpha_1 > \alpha_2 > \alpha_3$, які описані в підрозділі 2.3.3.

2.4.4. Вигорання, бар'єри та вплив дощу на згасання

Після того як клітина пройшла третю стадію горіння B_3 вона переходить до стану згорілої X . Згорілі клітини та бар'єри мають таку спільну характеристику: вони не змінюють свій стан на наступних кроках, тобто $X^t \rightarrow X^{t+1}$ і $R^t \rightarrow R^{t+1}$. Оскільки кандидатами на займання можуть бути тільки клітини в стані дерева, тому бар'єри не можуть брати участь у поширенні вогню.

Дощ також додатково прискорює згасання палаючих клітин. Наприклад для клітин у першій стадії горіння вводиться ймовірність переходу відразу до третьої стадії горіння. Ось формула за якою це відбувається:

$$P(B_1 \rightarrow B_3) = 0,25 \cdot R_t \quad (2.31)$$

А для клітин, які перебувають у другій стадії горіння вводиться ймовірність переходу одразу в стан згорілої клітини. Формула виглядає таким чином:

$$P(B_2 \rightarrow X) = 0,50 \cdot R_t \quad (2.32)$$

Якщо ж ці стохастичні переходи не відбуваються, то клітини переходитимуть до наступних станів за базовою схемою переходів, яка задана формулами (2.29)-(2.30).

2.5. Алгоритм кроку симуляції

На кожному кроці симуляції клітинний автомат синхронно оновлює стан усієї сітки. Ось яким чином проходить один такий крок.

Спочатку для поточної конфігурації G_t визначаються булеві маски клітин у кожному зі станів множини S , вона визначена формулою (2.4). Для стану B_1 маску можна формально записати таким чином:

$$\begin{aligned} M_{B_1}^t(i, j) &= 1, \text{ якщо } s_{i,j}^t = B_1 \\ M_{B_1}^t(i, j) &= 0, \text{ в іншому випадку} \end{aligned} \quad (2.33)$$

Маски станів для B_2, B_3, D, C, R, X визначаються аналогічно. Також ці маски використовуються на всіх наступних етапах кроку.

Після того, як маски сформовані на основі вхідних параметрів послідовно обчислюються наступні фактори середовища. Спочатку базова сухість D_h за формулою (2.10), далі нормована температура T_{norm} та температурний множник F_T , відповідно до формул (2.11)-(2.12), поточна інтенсивність дощу R_t за формулою (2.13) та ефективна сухість D_{eff} за формулою (2.15). Крім того, формується масив займистості $F_{veg}(i, j)$ відповідно до підрозділу 2.3.2, а також зчитуються коефіцієнти стадій горіння $\alpha_1, \alpha_2, \alpha_3$.

Наступний етап кроку визначає займання від сусідніх клітин. Тут для кожного напрямку $(\Delta i, \Delta j) \in N(i, j)$, визначеного формулою (2.5), перевіряється чи є наявні палаючі сусіди, обчислюється коефіцієнт F_{stage} за формулою (2.16) та вітровий множник $W(\Delta i, \Delta j)$ за формулами (2.21)-(2.22). Для клітин, яка є

потенційним кандидатом обчислюється ймовірність займання $P(\Delta i, \Delta j)$ за формулою (2.9), а також виконується стохастична перевірка згідно з умовою (2.23). Після отримання результатів з усіх восьми напрямків вони об'єднуються і якщо умова (2.23) виконалась хоча б раз (хоча б в одному з напрямків), то клітина вважатиметься, такою що загоряється.

Окремо обчислюється ймовірність події блискавки P_f за формулою (2.24), якщо така подія відбулась то вибираються клітини для її удару відповідно до формул (2.25)-(2.27). Після цього результати займання від сусідів та від блискавки об'єднуються операцією логічного АБО ($\vee$). На цьому ж етапі для клітин у стані горіння обчислюються переходи під дією дощу згідно з формулами (2.31)-(2.32).

На основі обчислених масок формується нова конфігурація G_{t+1} . Вона заповнюється порожніми клітинами E , після цього послідовно записуються стани відповідно до всіх описаних у підрозділі 2.4 правил переходів. На кінцевому етапі кроку поточна конфігурація змінюється на нову, також лічильник кроків збільшується на одиницю і до часового ряду $\{B_t\}$ додається поточна кількість клітин в станах B_1, B_2, B_3 .

Схематично алгоритм одного кроку симуляції можна подати таким чином:

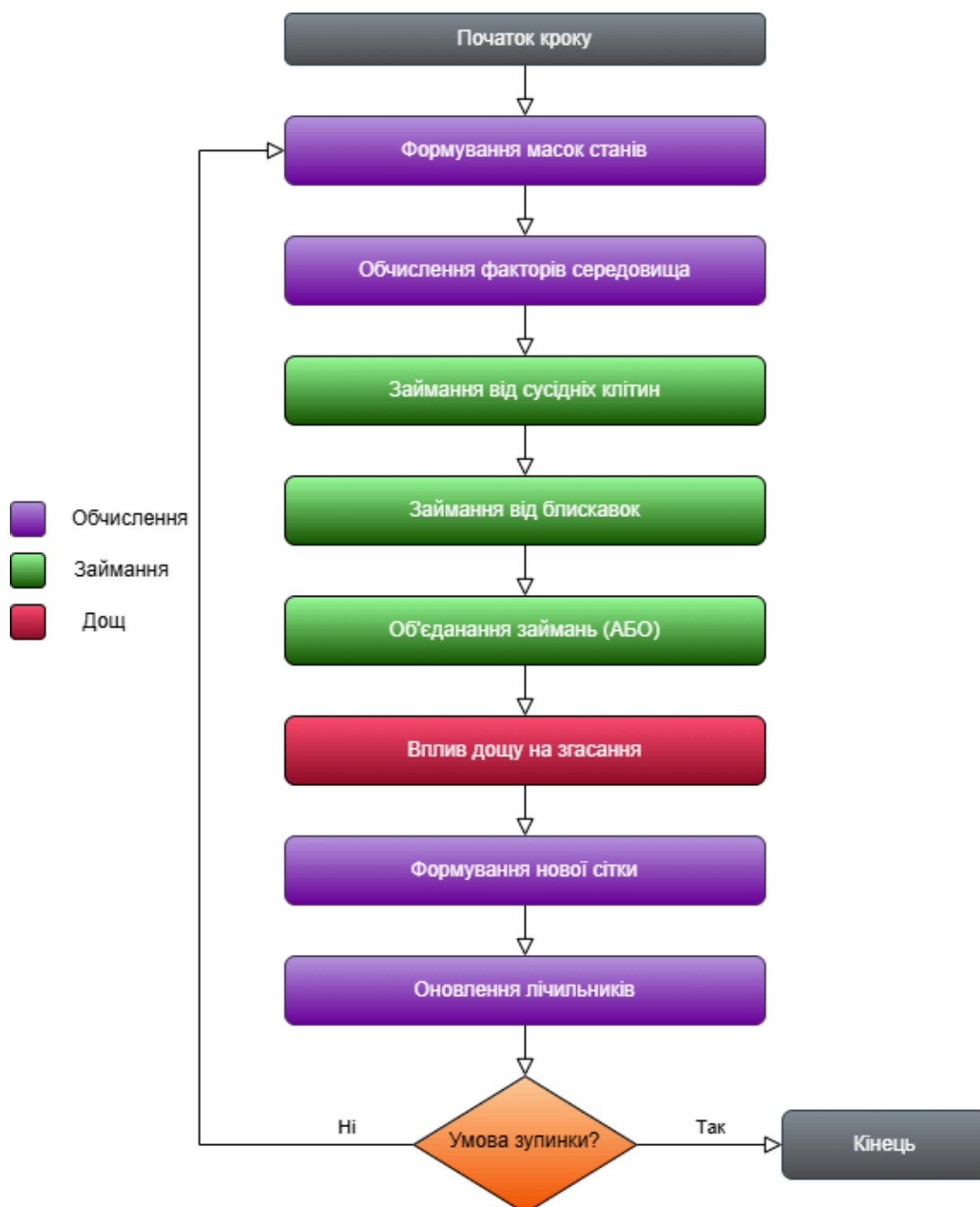


Рис 2.5.1 Схема одного кроку симуляції

Повний прогін симуляції виконується циклічним застосуванням кроку поки не виконається умова зупинки:

$$\left(\neg \exists (i, j) : s_{ij}^t \in \{B_1, B_2, B_3\} \right) \vee (t \geq T), \quad (2.34)$$

де T – максимальна кількість кроків симуляції. Якщо пожежа не згасла доки не вичерпався ліміт кроків, то прогін буде позначений як обрізаний, а часові метрики для такого прогону будуть правобічно цензурованими

2.6. Метрики оцінювання результатів

Для того, щоб кількісно оцінити результати симуляції використовується набір метрик, які показують масштаб, інтенсивність, динаміку та просторову структуру пожежі. Основним часовим рядом є B_t – кількість активних палаючих клітин в момент часу t . Також використовується N_0 та N_{burnt} , початкова кількість клітин в стані дерева та фінальна кількість згорілих клітин відповідно. Всі метрики також не прив'язуються до фізичних одиниць площі чи часу. Тепер перейдемо до формального опису метрик:

Масштаб пожежі описується як частка вигорілої площі BAF , що показує, яка частка дерев у результаті перейшла у стан вигорілих клітин:

$$\begin{aligned} BAF &= \min(1, N_{burnt}/N_0), \text{ якщо } N_0 > 0, \\ BAF &= 0, \text{ якщо } N_0 = 0. \end{aligned} \quad (2.35)$$

Інтенсивність пожежі характеризується такими двома показниками. Перший показник – максимальний розмір активного вогню:

$$Peak = \max\{B_t \mid t = 0, 1, \dots, T\} \quad (2.36)$$

Другим показником є інтегральна інтенсивність – загальна “активність” пожежі протягом всього прогону. Вона враховує не лише максимальний розмір фронту горіння, а й тривалість процесу. Визначається інтегральна інтенсивність як сума кількості палаючих клітин за всі кроки симуляції:

$$AUC = \sum_t B_t \quad (2.37)$$

Динаміка пожежі описується чотирма показниками. Час до піку горіння визначає перший момент часу, коли кількість палаючих клітин досягає свого максимального значення:

$$T_{peak} = \min\{t : B_t = \max\{B_k \mid k = 0, 1, \dots, K\}\} \quad (2.38)$$

Тривалість активного горіння характеризує загальну тривалість пожежі в межах симуляції. Цей показник визначається як кількість кроків, на яких хоча б одна клітина перебувала в стані горіння:

$$D_{fire} = |\{t : B_t > 0\}| \quad (2.39)$$

Час до згасання описується як перший момент після початку пожежі, коли кількість палаючих клітин стала нульовою:

$$T_{ext} = \min\{t : B_t = 0 \wedge \exists k < t : B_k > 0\} \quad (2.40)$$

Якщо ж пожежа до кінця прогону не згасла, то T_{ext} приймається як останній індекс часового ряду. Наступним показником є максимальна швидкість приросту активного вогню, він показує максимальне збільшення кількості палаючих клітин між двома сусідніми кроками часу, тобто найінтенсивніший момент розширення пожежі. Ось яким чином вона визначається:

$$R_{max} = \max(0, \max(B_t - B_{t-1} \mid t = 0, 1, \dots, T)) \quad (2.41)$$

Для того щоб коректно порівнювати прогони з різною початковою кількістю дерев потрібно використовувати нормовані метрики. Нормований пік пожежі:

$$PF = Peak/N_0, \text{ якщо } N_0 > 0, \\ PF = 0, \text{ якщо } N_0 = 0. \quad (2.42)$$

Нормована інтегральна інтенсивність:

$$AUC_{norm} = AUC/(N_0 \cdot L_B), \text{ якщо } N_0 \cdot L_B > 0, \\ AUC_{norm} = 0, \text{ в іншому випадку,} \quad (2.43)$$

де L_B – кількість точок часового ряду B_t .

Також окремо описується бінарна ознака критичного сценарію C_{crit} , яка показує, чи перевищила частка вигорілої площі заданий поріг критичності:

$$C_{crit} = [BAF \geq \theta], \quad (2.44)$$

де θ – заданий користувачем поріг критичності.

Щоб аналізувати просторову структура вигорілої зони використовують метрики, які обчислюються на основі маски згорілих клітин. Першою такою метрикою є C_{fire} , вона визначається як кількість 8-зв'язних компонент у масці згорілих клітин. Дві згорілі клітини вважаються пов'язаними, якщо вони є

сусідами згідно з сусідством Мура. Отже, якщо $C_{fire} = 1$, то вигоріла область є суцільною, якщо ж $C_{fire} > 1$, це означає, що пожежа утворила кілька окремих вигорілих ділянок.

Наступною метрикою є частка найбільшого вигорілого кластера, тобто показує, яку частину займає найбільша зв'язна компонента серед всієї вигорілої площі:

$$\begin{aligned} LCS &= A_{max} / A_{burnt}, \text{ якщо } A_{burnt} > 0, \\ LCS &= 0, \text{ якщо } A_{burnt} = 0, \end{aligned} \quad (2.45)$$

де A_{max} – площа найбільшої зв'язної компоненти вигорілої зони;

A_{burnt} – загально площа вигорілої зони.

Останньою метрикою є складність форми вигорілої зони:

$$\begin{aligned} SC &= P_{4n} / A_{burnt}, \text{ якщо } A_{burnt} > 0, \\ SC &= 0, \text{ якщо } A_{burnt} = 0, \end{aligned} \quad (2.46)$$

де P_{4n} – периметр вигорілою зони, що визначається як кількість відкритих ребер у сусідів з чотирьох сторін (ортогональні сусіди) між згорілими та незгорілими клітинами.

Чим більшим буде значення параметра SC , тим більш нерівномірною буде форма вигорілої зони.

3. ПРОГРАМНА РЕАЛІЗАЦІЯ СИМУЛЯТОРА ЛІСОВИХ ПОЖЕЖ

3.1. Вибір технологічного стеку та обґрунтування

Вибір інструментів розробки є важливим етапом проєктування програмного застосунку тому, що він прямо впливає на продуктивність обчислень, підтримуваність коду та якість кінцевого продукту. Тому обираючи технологічний стек для проєкту, в даному випадку для симулятора лісових пожеж враховувались наступні критерії. Важливими ознаками є відповідність інструментів задачам

наукового програмування, обчислювальна ефективність при роботі з двовимірними масивами даних, наявність інструментів для побудови графічного інтерфейсу та аналітичної підсистеми, а також відкритість і доступність усіх складових частин. Детальніший опис використаних технологій буде описано у наступних розділах, а загальний перелік наведено у таблиці 3.1.

Технологія	Призначення
Python	Основна мова реалізації всього застосунку
NumPy	Сітка клітинного автомата, векторизовані обчислення, стохастика
PySide6 (Qt)	Графічний інтерфейс користувача
Matplotlib	Аналітичні графіки для звітів
Pandas	Збереження результатів у форматі Parquet
pytest	Тестування коректності ядра симулятора
Ruff / Black	Статичний аналіз та форматування коду

Табл. 3.1 – Технологічний стек застосунку

3.1.1. Python

В даному застосунку Python є основною мовою програмування, якою реалізовано ядро клітинного автомата, графічний інтерфейс, підсистему серійних експериментів, обчислення метрик та формування результатів моделювання. Ця мова є однією з найпоширеніших для використання у науковому програмуванні, завдяки наступним причинам. Лаконічний та простий синтаксис, він значно спрощує реалізацію математичної моделі та правил переходів клітинного автомата. В межах цієї роботи це особливо важливо, оскільки модель містить значну кількість параметрів, а Python дозволяє подати ці правила у досить компактному вигляді і при тому не погіршуючи читабельність коду.

Крім того, Python має велику кількість спеціалізованих бібліотек для чисельних обчислень, обробки даних, побудові графіків та створенні графічних

інтерфейсів. Тут ця його перевага застосовується безпосередньо. Для роботи з масивами використовується NumPy, для табличної обробки результатів – Pandas, для побудови графіків – Matplotlib, а для створення десктоп-інтерфейсу – PySide6. Завдяки наявності цих технологій всі частини симулятора реалізовані однією мовою програмування.

Також Python добре підходить для реалізації імітаційних моделей, в яких є важливою не тільки швидкість розробки, а й можливість інтеграції з аналітичними інструментами. Для даного проєкту використовується версія Python 3.12, вона забезпечує підтримку всіх використаних бібліотек та передовий функціонал мови. Вхідною точкою десктопного застосунку є файл `src/app/main.py`, а для запуску серійних експериментів використовується окремий скрипт `run_experiments.py`.

3.1.2. NumPy

NumPy – це фундаментальна бібліотека Python для роботи з багатовимірними масивами та виконання чисельних обчислень. У моделі клітинного автомата стан лісового середовища подається як двовимірний масив, де кожен елемент відповідає окремій клітині сітки, а також містить числовий код її поточного стану.

Ключовою перевагою NumPy для задач моделювання є концепція векторизації – можливість виконувати операції одночасно над усіма елементами масиву без явних циклів. В програмі для цього використовуються булеві маски, що дозволяють швидко визначити якого типу клітина. Далі ці маски застосовуються для обчислення претендента на займання, переходів між стадіями горіння та для підрахунку статистики.

Також NumPy використовується для впровадження стохастичних процесів у моделі. Наприклад, генератор випадкових чисел `np.random.default_rng` застосовується для початкової генерації лісу, випадкового займання від сусідніх клітин та ударів блискавки, а також впливу дощу на згасання пожежі. Ще використовується параметр `seed`, що забезпечує відтворюваність результатів.

Завдяки цій бібліотеці можна виконувати значну частину операцій одразу над усією сіткою, а не обробляти кожен клітинний звичайним циклом Python. Це звісно в разі підвищує швидкість роботи симулятора, особливо на великих сітках.

3.1.3. PySide6 (Qt)

Бібліотека PySide6 використовується для реалізації графічного інтерфейсу користувача цього симулятора. Вона є офіційними Python-прив'язками до кросплатформеного фреймворку Qt і надає можливість створювати повноцінні десктоп застосунки з вікнами, віджетами, панелями керування та обробкою подій користувача.

В цьому проєкті PySide6 застосовується для побудови головного вікна застосунку, елементів керування параметрами, області візуалізації сітки клітинного автомата та інструментів для інтерактивного редагування середовища. Через графічний інтерфейс користувач може повноцінно керувати симуляцією, змінювати параметри та вручну редагувати клітини на сітці. Також перевагою цієї бібліотеки є підтримка подієвої моделі Qt, тобто обробка натискань на кнопки, зміни значень параметрів, взаємодія миші з клітинами сітки та керування таймером симуляції. Завдяки цьому автоматичний режим реалізується як послідовне виконання кроків клітинного автомата з певним часовим інтервалом, а покроковий режим дає вручну спостерігати за зміною станів.

Також PySide6 дає можливість подати сітку у графічному вигляді, де кожному стану відповідає окремий колір, що наочно ілюструє симуляцію.

Вибір саме цієї бібліотеки для симулятора є найбільш доцільним, оскільки він має бути не лише обчислювальною моделлю, а й інтерактивним застосунком. Використання Qt дає поєднати в одному графічному середовищі керування параметрами, візуалізацію клітинної сітки та інструменти для редагування середовища. У структурі застосунку компоненти інтерфейсу є відокремленими від ядра моделі, тобто PySide6 відповідає переважно за взаємодію з користувачем та відображення результатів, а обчислення вже виконуються в модулі симуляції.

3.1.4. Matplotlib

Matplotlib є комплексною бібліотекою Python для створення статичних і інтерактивних наукових графіків та візуалізацій. Конкретно в цьому проекті вона не використовується в основному вікні симулятора, а в підсистемі пост-аналізу серійних експериментів. Зокрема, за допомогою Matplotlib будуються гістограми розподілу метрик, діаграми для порівняння різних сценаріїв, теплові карти залежностей між параметрами та часові ряди активності пожежі. Всі графіки автоматично зберігаються у складі згенерованих звітів у форматах Markdown (спрощена мова розмітки для оформлення текстових документів) та HTML, що дозволяє переглядати результати експериментів без запуску застосунку.

3.1.5. Pandas

Pandas – це бібліотека Python для роботи з табличними даними і застосовується в підсистемі серійних експериментів для збереження зібраних метрик у форматі Parquet. Цей формат допомагає ефективно стискати дані та швидко їх зчитувати для подальшого аналізу великих наборів результатів. Pandas в проекті є опційною залежністю, тобто якщо бібліотека буде недоступна в середовищі виконання, то застосунок буде зберігати лише у форматі CSV, а не аварійно завершувати роботу. Це дозволяє гнучко розгортати застосунок в різних середовищах.

3.1.6. Інструменти якості коду

Щоб забезпечити якість та узгодженість коду в проекті використовуються наступні два інструменти. Першим є Ruff – сучасний швидкий аналізатор коду, написаний мовою Rust, він виявляє потенційні помилки, невикористані імпорти та порушення стилю. Наступним є Black – це інструмент автоматичного форматування коду, він забезпечує єдиний стиль оформлення у всіх файлах проекту. На основі pytest побудована тестова інфраструктура застосунку, оскільки він є стандартом тестування у Python і дозволяє перевіряти коректність роботи ядра симулятора на граничних і типових випадках. Усі залежності проекту

зафіксовані у файлі requirements.txt, що дозволяє відтворювати середовище розробки та виконання.

3.2. Архітектура застосунку

Симулятор лісових пожеж побудовано за модульним принципом з розділенням на три незалежні шари: ядро моделі (папка core), графічний інтерфейс (папка ui) та підсистема для серійних експериментів (папка experiments). Кожен з цих шарів реалізований як окремий пакет в межах директорії src/app/. Завдяки такій організації коду компоненти менш зв'язані між собою, що в разі спрощує тестування і дозволяє розширювати функціональність без змін в інших шарах. Загальна структура проекту зображена на рис. 3.2.1:

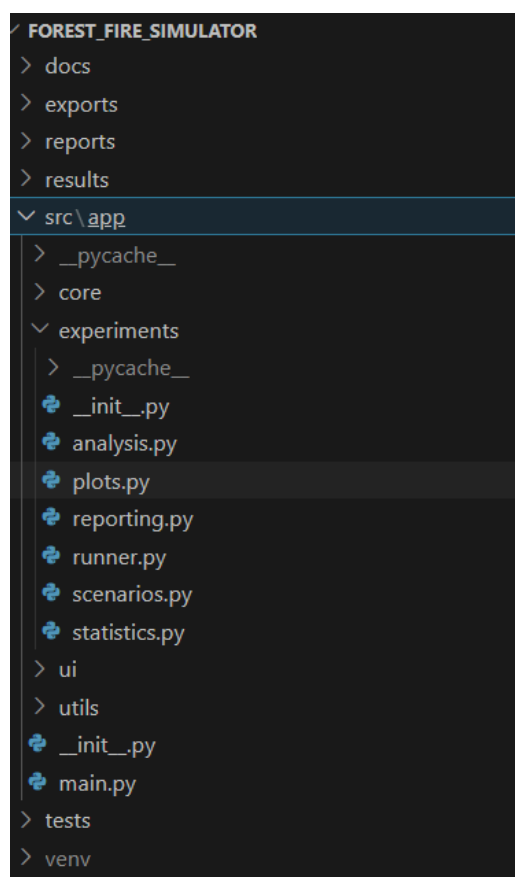


Рис. 3.2.1 – Загальна структура проекту

3.2.1. Ядро моделі

Шар ядра моделі (core) містить всю логіку клітинного автомата і він є незалежним від графічного інтерфейсу, тобто його можна використовувати

окремо, наприклад з командного рядка. Структура шару core зображена на рис.

3.2.1.1. Цей шар складається з наступних модулів:

- `constants.py` – модуль, який визначає числові коди станів клітин сітки таких, як порожня клітина, листяне/хвойне дерево, три стадії горіння, бар'єр та вигоріла клітина. Тут використовуються іменовані константи замість числових літералів, що підвищує читабельність коду та спрощує внесення змін до множини станів.
- `config.py` – цей модуль містить клас `CAConfig`, який об'єднує всі параметри, що йдуть на вхід моделі: розміри сітки, густоту та склад лісу, коефіцієнти займистості, погодні умови, параметри блискавки та зерно генератора випадкових чисел. Завдяки цьому модулю є можливість передавати всю конфігурацію як єдиний об'єкт, а це спрощує створення і відтворення різних сценаріїв симуляції.
- `engine.py` – модуль, що містить `ForestFireCA`, саме цей клас є центральним компонентом всього застосунку. Він реалізує генерацію початкового стану лісу, векторизований крок симуляції застосунку `step()`, обчислення погодних факторів і вітрового множника, стохастичне займання від сусідніх клітин і блискавок, переходи між стадіями горіння, а також методи для інтерактивного редагування середовища.
- `metrics.py` та `spatial_metrics.py` – ці два модулі реалізують обчислення кількісних показників результатів симуляції. `metrics.py` відповідає за часові метрики такі, як частка вигорілої площі, тривалість пожежі, час до піку та час до згасання. А модуль `spatial_metrics.py` обчислює просторові характеристики вигорілої зони, а саме кількість зв'язних компонентів, частку найбільшого кластера та складність форми.

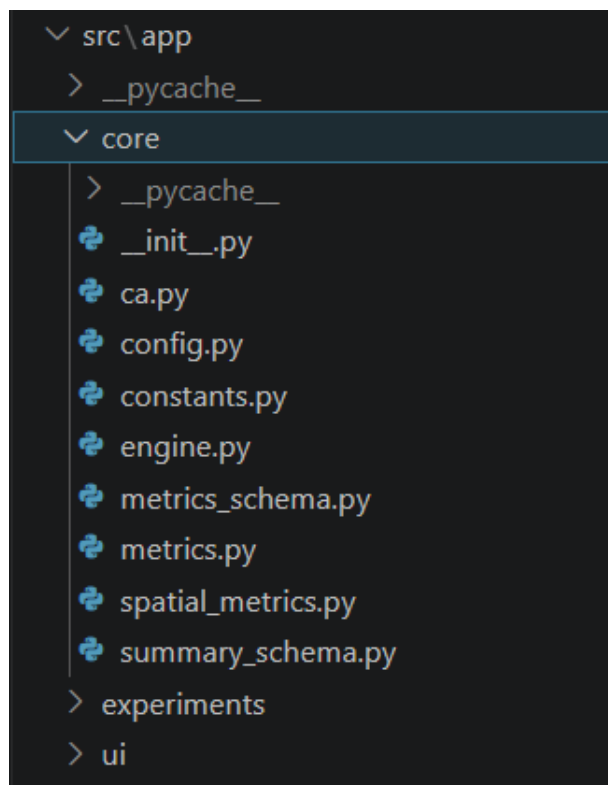


Рис. 3.2.1.1 – Структура шару core

3.2.2. Графічний інтерфейс

Шар ui реалізує весь графічний інтерфейс на основі бібліотеки PySide6 і складається з таких модулів:

- `main_window.py` – визначає клас головного вікна `MainWindow`, який є підкласом `QMainWindow`. У конструкторі головного вікна створюється екземпляр `ForestFireCa` і тут же ініціалізуються всі дочірні віджети, панелі та таймер автоматичного режиму симуляції.
- `grid_widget.py` – це модуль, що реалізує віджет для відображення сітки клітинного автомата. Також він перевизначає метод `paintEvent()` для безпосереднього рендерингу масиву станів `NumPy` на полотні `QWidget`, що дозволяє відображати кожен стан окремим кольором.
- `main_window_actions.py` – модуль, що реалізовано як `mixin`-клас `MainWindowActionsMixin`, тобто такий допоміжний клас, який додає основному класу набір методів, але не використовується як самостійний об'єкт. Він містить всі обробники подій: кнопок керування симуляцією, таймера автоматичного режиму, інструментів редагування середовища та

функції експорту. Обробники безпосередньо викликають методи об'єкта клітинного автомата, а після виконання змін оновлюють `grid_widget` для того, щоб відобразити актуальний стан сітки.

- `panels/` – це директорія, що також міститься в шарі `ui` і містить модулі панелей керування параметрами симуляції та блоками відображення поточних метрик. Кожна панель є окремим класом, що наслідує `QWidget`.

Структура шару `ui` зображена на рис. 3.2.2.1.

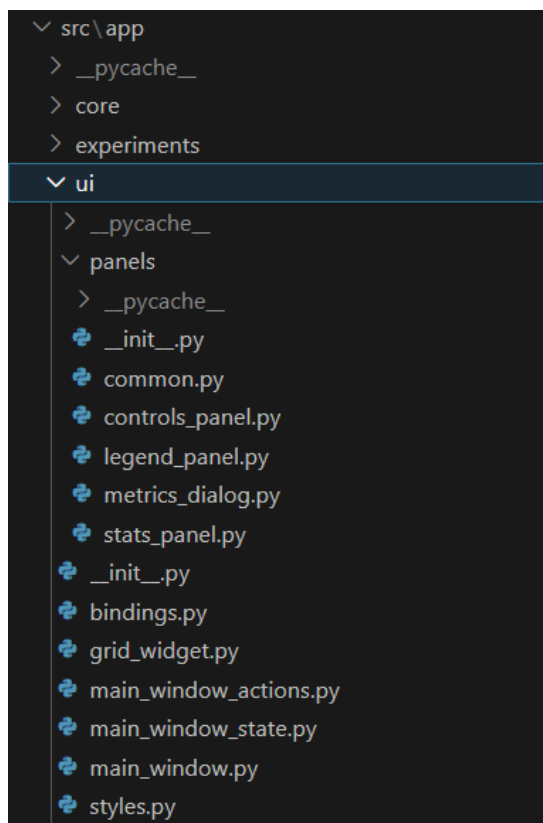


Рис. 3.2.2.1 – Структура шару `ui`

3.2.3. Серійні експерименти та аналітика

Шар `experiments` забезпечує пакетне виконання симуляції та аналіз їх результатів і складається з модулів, що наведені нище:

- `runner.py` – модуль, який реалізує багаторазові прогони симуляції з різними зернами випадкових чисел. Для кожного прогону створюється окремий екземпляр `ForestFireCA`, виконується повна симуляція до умови зупинки і збираються метрики. Результати зберігаються у форматах `CSV` та `Parquet`.

- `analysis.py` – цей модуль відповідає за агрегування результатів серійних експериментів, тобто за об'єднання даних з багатьох запусків симуляції в узагальнену статистику. Тут проходять обчислення середніх значень, стандартних відхилень та аналіз чутливості метрик до зміни параметрів середовища.
- `reporting.py` – генерує підсумкові звіти в форматах Markdown та HTML на основі зібраних метрик та графіків.
- `plots.py` – модуль, який будує аналітичні графіки за допомогою бібліотеки Matplotlib, наприклад гістограми розподілу метрик, діаграми розмаху, теплові карти залежностей між параметрами та часові ряди активності горіння.

Структура шару `experiments` зображена на рис. 3.2.3.1.

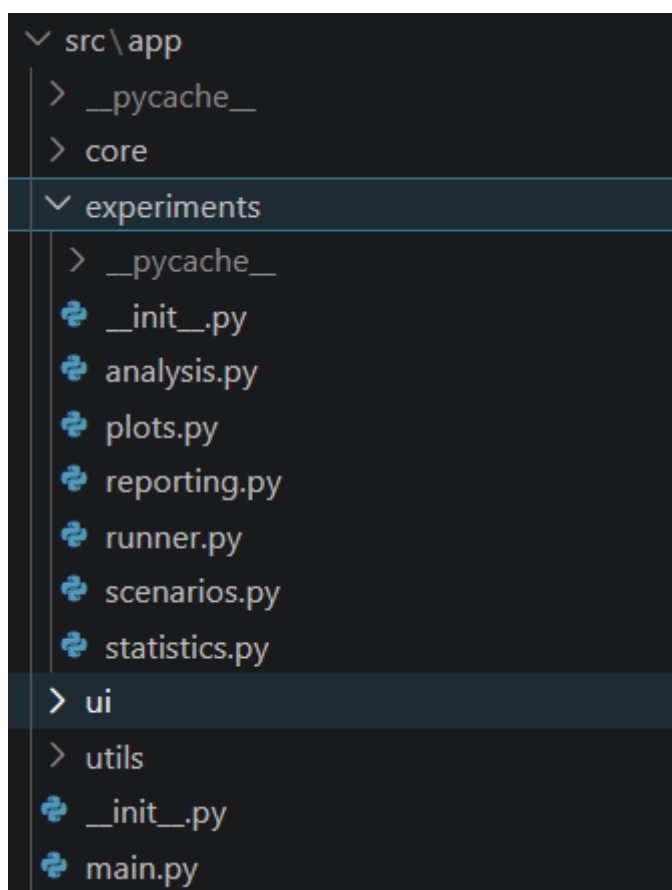


Рис. 3.2.3.1 – Структура шару `experiments`

3.2.4. Точки входу та взаємодія між шарами

Застосунок має дві незалежні точки входу. Першою точкою є файл `src/app/main.py`, який запускає GUI-застосунок, тобто ініціалізує об'єкт `QApplication`, створює головне вікно та передає керування циклу подій `Qt`. `Qt` – кросплатформений фреймворк для розробки настільних застосунків, який забезпечує роботу з вікнами, віджетами, подіями, таймерами, графічним відображенням і використовується в даному проєкті через бібліотек `PySide6`. Другою точкою входу є скрипт `run_experiments.py`, який використовується для запуску серійних експериментів через командний рядок без графічного інтерфейсу. Це потрібно для автоматизованого виконання великої кількості прогонів симуляції, збору метрик, порівняння сценаріїв та формування звітів.

Взаємодія між шарами організована наступним чином. Шар графічного інтерфейсу (`ui`) безпосередньо викликає методи класу `ForestFireCA` з шару ядра моделі (`core`) і після кожного виклику оновлює `grid_widget` для того, щоб відобразити актуальний стан сітки. Шар `experiments` також безпосередньо використовує клас `ForestFireCA` для виконання прогонів, після цього передає зібрані метрики до модулів `analysis`, `reporting` та `plots`. Крім того, важливо зазначити, що шар `core` від жодного іншого шару застосунку і це дозволяє використовувати його незалежно.

3.3. Реалізація ядра симулятора

Ядро симулятора реалізовано в модулі `engine.py` у вигляді класу `ForestFireCA`. Цей клас повністю інкапсулює стан сітки клітинного автомата та реалізує математичну модель, яка описана в розділі 2. Клас приймає на вхід об'єкт конфігурації, що містить всі вхідні параметри моделі. У конструкторі ініціалізується генератор випадкових чисел з заданим зерном, після чого викликається метод для формування початкової конфігурації лісу. Також ініціалізується лічильник кроків, історія кількості палаючих клітин та словники збереження фінальних метрик. Структура конструктора та метод генерації початкового лісу зображена на рис. 3.3.1.

```

engine.py src/app/core/engine.py ForestFireCA
23 class ForestFireCA:
24     def __init__(self, cfg: CAConfig):
25         self.cfg = cfg
26         self.rng = np.random.default_rng(cfg.seed)
27         self.grid = self._make_initial_grid()
28         self.step_count = 0
29         self.lightning_cooldown = 0
30         self.initial_tree_cells = 0
31         self.burning_cells_history: list[int] = []
32         self.final_counts: dict[str, int] = {}
33         self.latest_metrics: dict[str, int | float] = {}
34         self.start_run_tracking()
35
36     def _make_initial_grid(self) -> np.ndarray:
37         cfg = self.cfg
38         h, w = cfg.height, cfg.width
39         grid = np.full((h, w), EMPTY, dtype=np.uint8)
40
41         has_tree = self.rng.random((h, w)) < float(np.clip(cfg.init_tree_density, 0.0, 1.0))
42         conifer_ratio = float(np.clip(cfg.conifer_ratio, 0.0, 1.0))
43         is_conifer = has_tree & (self.rng.random((h, w)) < conifer_ratio)
44         is_decid = has_tree & ~is_conifer
45
46         grid[is_decid] = TREE_DECID
47         grid[is_conifer] = TREE_CONIF
48         return grid
49

```

Рис. 3.3.1 – Конструктор класу ForestFireCA та метод _make_initial_grid()

Основним методом класу є step(), він реалізує один крок симуляції відповідно до алгоритму підрозділу 2.5. Початок методу step() із формуванням масок та обчисленням факторів середовища зображено на рис. 3.3.2.

```

engine.py src/app/core/engine.py ForestFireCA _lightning_event
23 class ForestFireCA:
272 def step(self):
273     g = self.grid
274
275     b1 = (g == BURNING1)
276     b2 = (g == BURNING2)
277     b3 = (g == BURNING3)
278
279     decid = (g == TREE_DECID)
280     conifer = (g == TREE_CONIF)
281     is_tree = decid | conifer
282
283     barrier = (g == BARRIER)
284     burnt = (g == BURNT)
285
286     humidity = float(np.clip(self.cfg.humidity, 0.0, 1.0))
287     dryness = 1.0 - humidity
288
289     t_norm = self._temp_norm()
290     temp_factor = 0.5 + t_norm
291
292     rain = self.current_rain_intensity()
293     dryness_eff = float(np.clip(dryness * temp_factor * (1.0 - rain), 0.0, 1.0))
294
295     flamm = np.zeros(g.shape, dtype=np.float32)
296     flamm[decid] = float(np.clip(self.cfg.flamm_decid, 0.0, 5.0))
297     flamm[conifer] = float(np.clip(self.cfg.flamm_conifer, 0.0, 5.0))
298

```

Рис. 3.3.2 – Фрагмент методу step()

3.4. Графічний інтерфейс користувача

Графічний інтерфейс застосунку реалізовано з використанням PySide6. Головне вікно поділене на дві зони (рис. 3.4.1), у лівій зоні розташоване поле симуляції – віджет відображення сітки клітинного автомата, де кожна клітина відображається певним кольором відповідно до свого стану. Під полем розташована легенда з колірним кодуванням для всіх станів: порожньо, листяне дерево, хвойне дерево, три стадії горіння, бар'єр та випалено.



Рис. 3.4.1 – Головне вікно симулятора

Права зона містить панель керування з блоком огляду та трьома вкладками для редагування параметрів симуляції. Блок огляду відображає поточний крок симуляції, кількість палаючих клітин, кількість живих дерев та статус дощу в реальному часі. Вкладка “Загальні” містить елементи керування відтворенням – кнопки старт, пауза, крок і скинути мапу, випадний список вибору активного інструменту редагування, а також поля налаштування розміру сітки та панель для регулювання швидкості відтворення симуляції. Вкладка “Середовище” надає

доступ до погодних умов через слайдери сили вітру, вологості, температури та налаштування сценаріїв дощу (рис. 3.4.2).

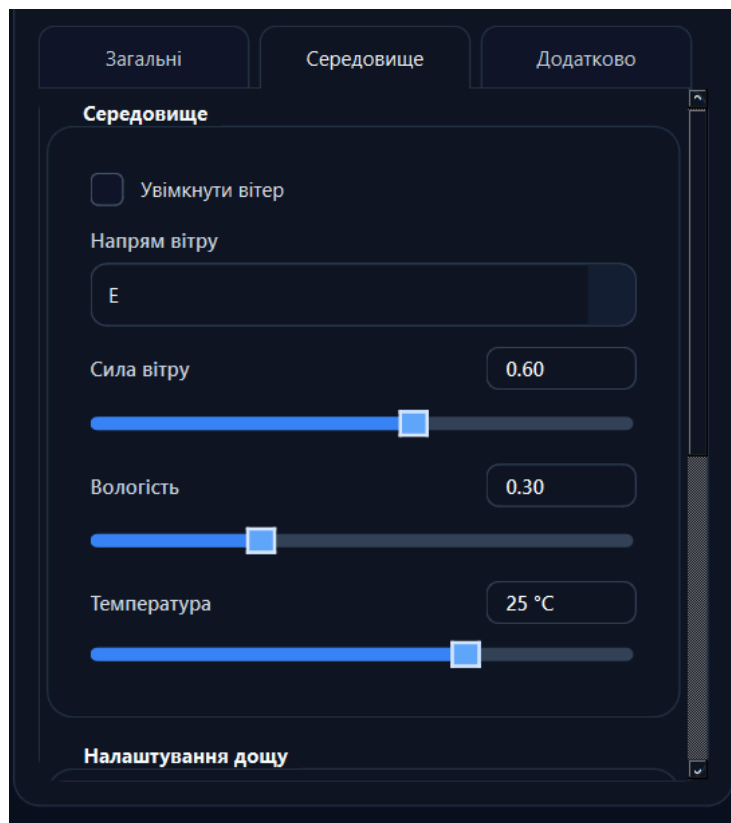


Рис. 3.4.2 – Вкладка “Середовище”

Вкладка “Додатково” містить налаштування займистості дерев та параметрів блискавки (рис. 3.4.3).

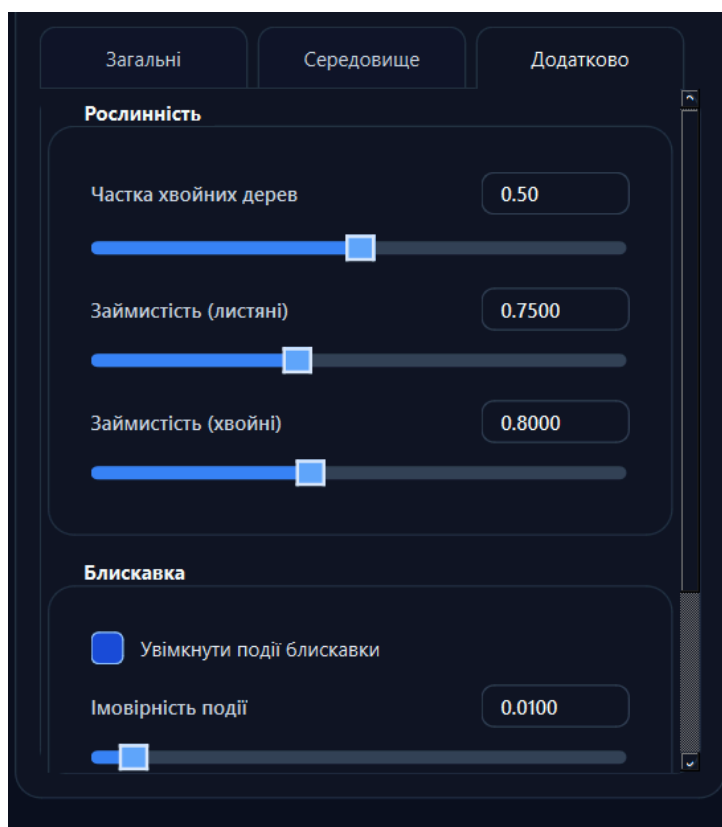


Рис. 3.4.3 – Вкладка “Додатково”

Для інтерактивного редагування середовища користувач обирає один з п'яти інструментів з випадного списку: підпал, посадити листяне дерево, посадити хвойне дерево, бар'єр або стерти (рис. 3.4.4). Лівою кнопкою миші застосовується активний інструмент, щоб змінити стан клітин, тобто користувач може як натиснути на окрему клітину, так і затиснувши кнопку миші провести по сітці, змінюючи стан усіх клітин, через які проходить курсор. Права кнопка миші використовується для швидкого стирання стану клітин незалежно від обраного інструменту. Редагування доступне лише в режимі паузи.

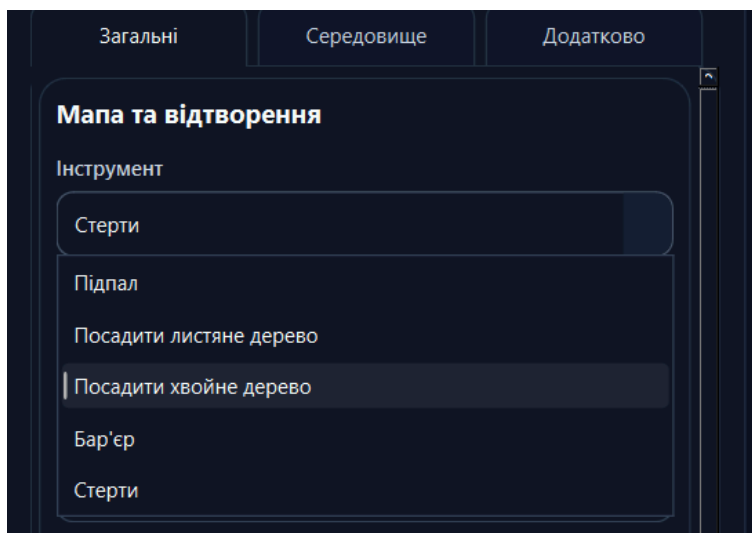


Рис. 3.4.4 – Випадний список інструментів редагування

Кнопка “Відкрити аналітику” стає доступною після завершення прогону і відкриває окреме вікно з фінальними метриками симуляції (рис. 3.4.5). До них належать: BAF, що характеризує частку вигорілої площі; Peak fire size, що показує максимальну кількість одночасно палаючих клітин; Time to peak, яка визначає момент досягнення пікової інтенсивності пожежі; Fire duration, що відображає тривалість активного горіння; та AUC, який узагальнює інтенсивність пожежі протягом всього прогону. Кнопка “Експорт метрик” зберігає показники у файл формату JSON.

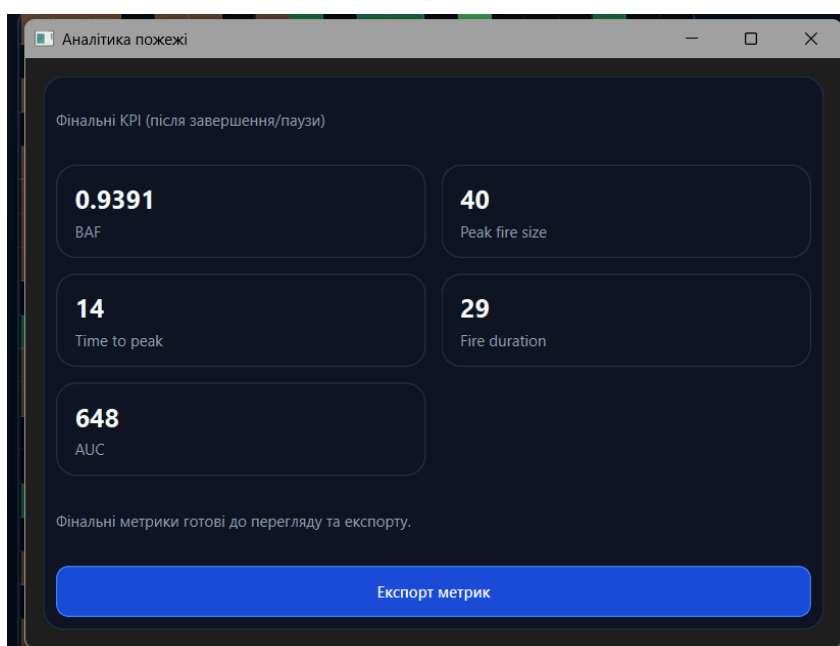
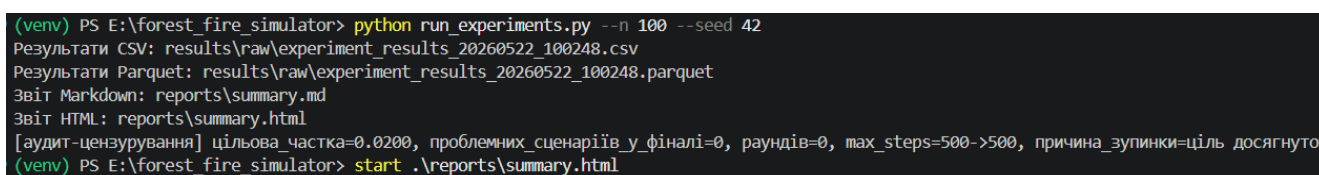


Рис. 3.4.5 – Вікно з фінальними метриками

3.5. Серійні експерименти, збір метрик та експорт результатів

Підсистема серійних експериментів реалізована у шарі `experiments` і запускається через CLI-скрипт `run_experiments.py`. CLI – це інтерфейс командного рядка, який дозволяє запускати експерименти без відкриття графічного вікна. Основні параметри запуску включають кількість прогонів на сценарій (`--n`), базове зерно генератора випадкових чисел (`--seed`), максимальну кількість кроків симуляції (`--max-steps`), поріг критичності (`--critical-baf-threshold`) та директорії збереження результатів і звітів (`--results-dir`, `reports-dir`). Приклад запуску серійних експериментів зображено на рис. 3.6.1.



```
(venv) PS E:\forest fire simulator> python run_experiments.py --n 100 --seed 42
Результати CSV: results\raw\experiment_results_20260522_100248.csv
Результати Parquet: results\raw\experiment_results_20260522_100248.parquet
Звіт Markdown: reports\summary.md
Звіт HTML: reports\summary.html
[аудит-цензурування] цільова_частка=0.0200, проблемних_сценаріїв_у_фіналі=0, раундів=0, max_steps=500->500, причина_зупинки=ціль_досягнута
(venv) PS E:\forest fire simulator> start .\reports\summary.html
```

Рис. 3.6.1 – Запуск серійних експериментів через CLI

Пайплайн одного прогону реалізовано в модулі `runner.py`. Під пайплайном у цьому випадку розуміється послідовність етапів, які автоматично виконуються під час одного запуску симуляції: від завантаження параметрів до збереження результатів. Він включає завантаження конфігурації сценарію, створення екземпляра `ForestFireCA` із заданими параметрами та зерном, ініціалізація займання, циклічне виконання кроків `step()` до повного згасання пожежі або досягнення ліміту кроків. Якщо пожежа не згасла до завершення цього ліміту, то у результатах окремо фіксується, що прогін був примусово зупинений через досягнення максимальної кількості кроків. Результат кожного прогону зберігається, як окремий запис, який містить ідентифікатор прогону, назву сценарію, використане зерно генератора випадкових чисел, параметри конфігурації та обчислені метрики.

Після завершення кожного прогону обчислюються метрики, які описані в розділі 2.6. Вони характеризують, масштаб пожежі, її часову динаміку, інтегральну інтенсивність, швидкість поширення та просторову структуру вигорілої зони. Отримані показники далі використовуються для порівняння сценаріїв та статичного аналізу результатів серійних експериментів. Для кращої

інтерпретації результати аналізуються у двох варіантах: зі всіма виконаними прогонами та окремо з тими, які закінчились природним згасанням пожежі до досягнення максимальної кількості кроків. Це дозволяє не змішувати самостійно завершені прогони із примусово зупиненими.

Результати зберігаються у двох форматах: CSV зберігається завжди, Parquet – за наявності бібліотеки Pandas. Імена файлів містять UTC-мітку часу для забезпечення трасованості, тобто можливості одночасно пов'язати кожен файл результатів із конкретним запуском експерименту. Це дозволяє перевірити в майбутньому коли було виконано запуск, які в нього результати та звіт. Після завершення всіх прогонів модулі `analysis.py`, `reporting.py` та `plots.py` автоматично агрегують результати, обчислюють статистики та генерують підсумковий звіт у форматах Markdown та HTML. Структуру згенерованих файлів звіту зображено на рис. 3.6.2, а фрагмент HTML-звіту з загальним підсумком – на рис. 3.6.3.

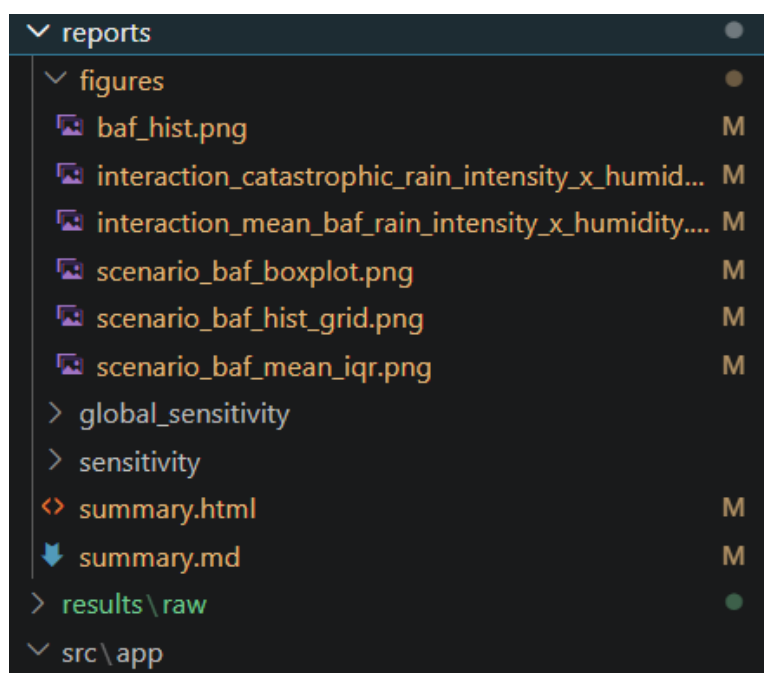


Рис. 3.6.2 – Структура згенерованих файлів звіту

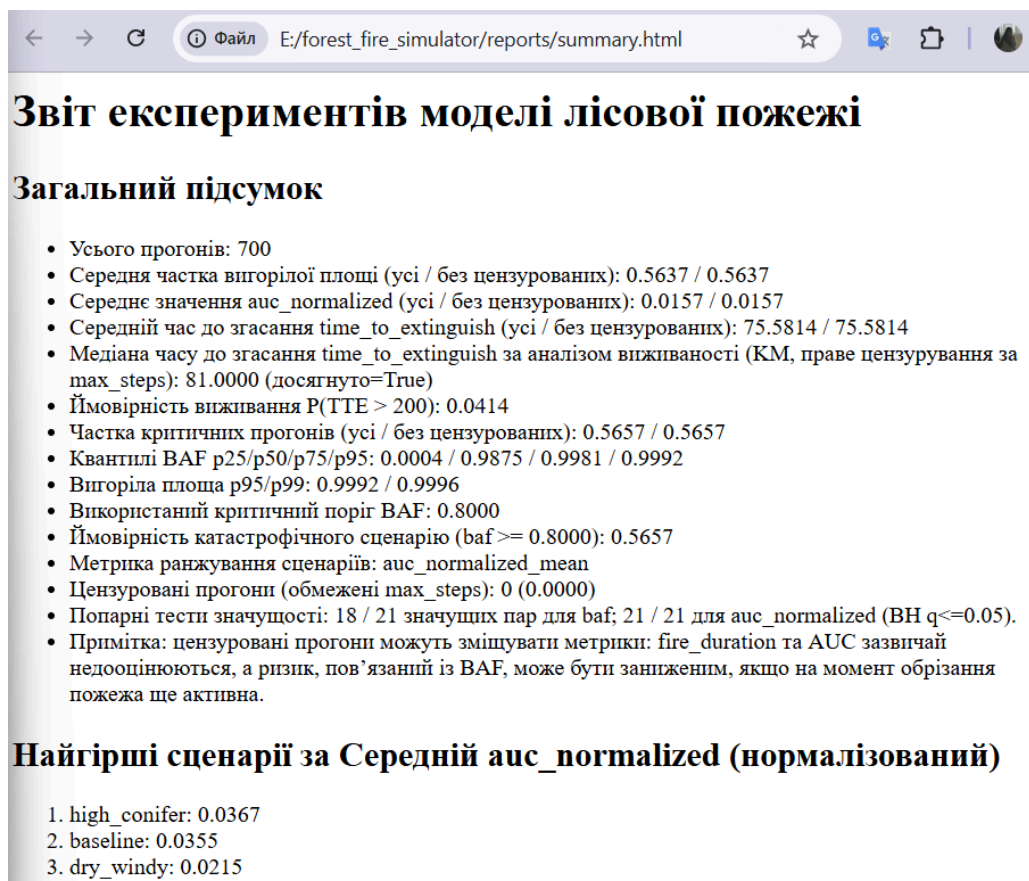


Рис. 3.6.3 – Фрагмент згенерованого HTML-звіту із загальним підсумком

Підсистема також підтримує два типи аналізу чутливості. OFAT-аналіз (One Factor At a Time), тобто змінює один параметр за раз відносно базових умов для визначення локальних тенденцій. Глобальний аналіз чутливості одночасно варіює кілька параметрів і будує двовимірні карти взаємодій між парами факторів. Для кожного типу аналізу генеруються графіки. Для прикладу наведено лише два види графіків: на рис. 3.6.4 гістограма розподілу BAF по всіх прогонах, а на рис. 3.6.5 діаграма розмаху для порівняння сценаріїв.

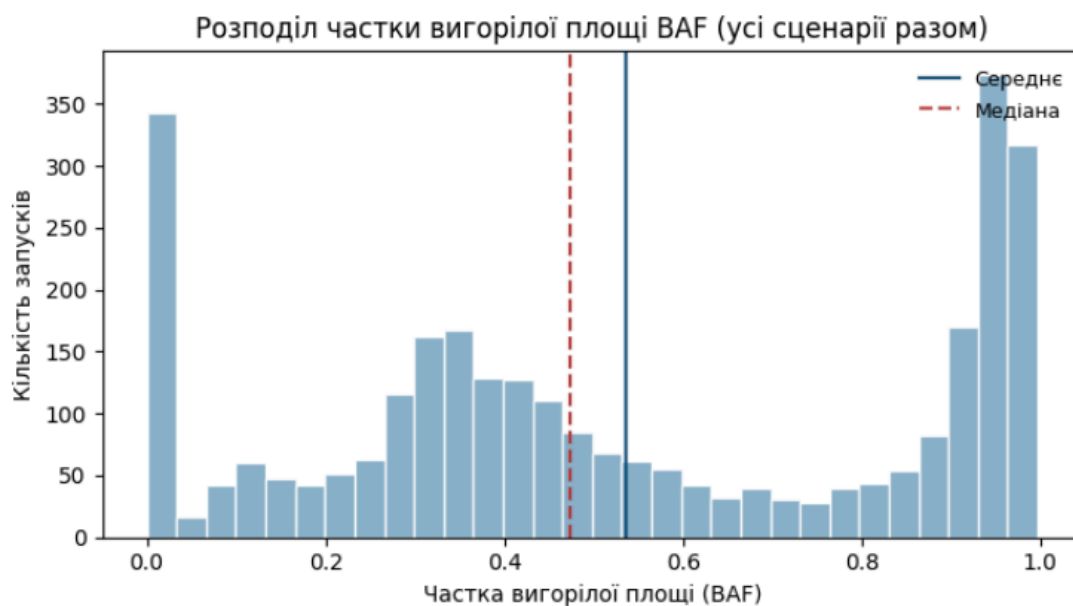


Рис. 3.6.4 – Гістограма розподілу частки вигорілої площі BAF

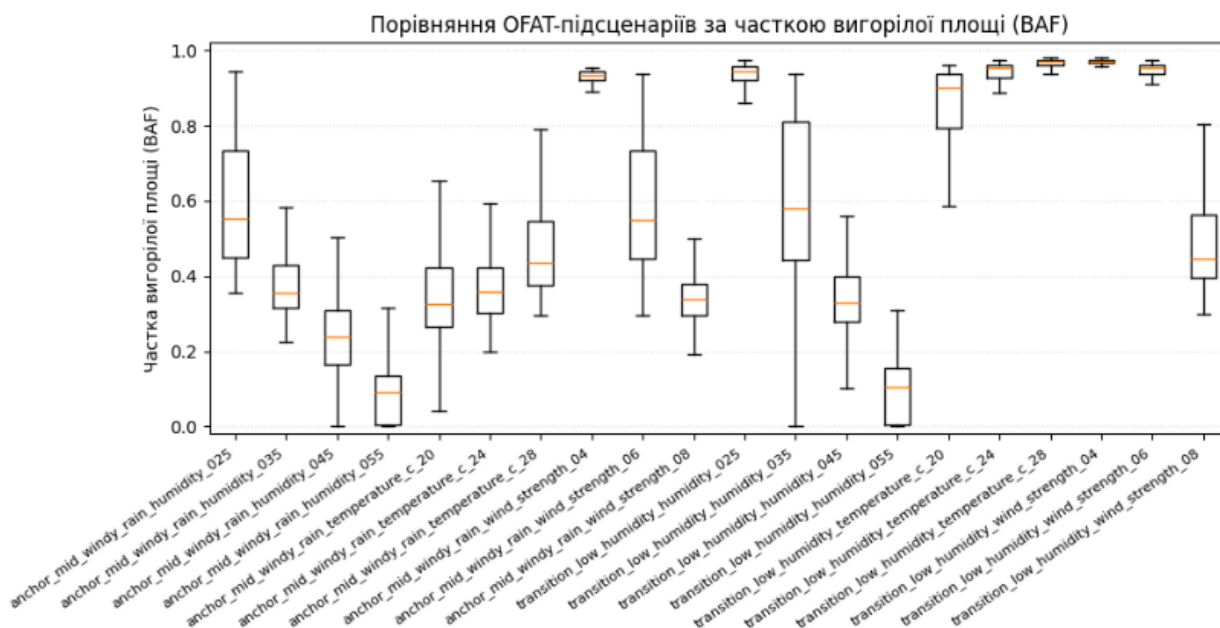


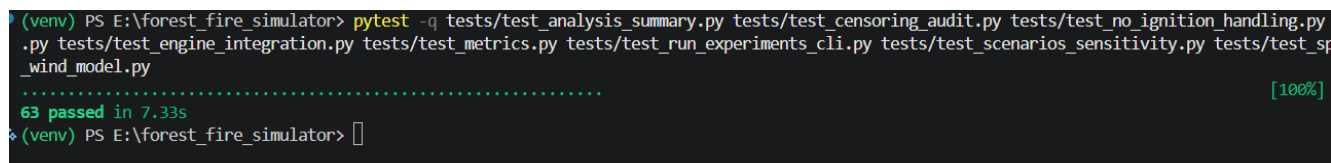
Рис. 3.6.5 – Порівняння OFAT-підсценаріїв за часткою вигорілої площі BAF

3.6. Тестування та відтворюваність результатів

Відтворюваність результатів симуляції забезпечується на двох рівнях. На рівні окремого прогону використовується генератор випадкових чисел із фіксованим зерном. Це гарантує однакову початкову конфігурацію та однакову послідовність стохастичних подій при однакових вхідних параметрах. А на рівні серійних експериментів зерна всіх прогонів генеруються детерміновано від

базового зерна, заданого параметром `--seed`, що дозволяє точно відтворити будь-який набір результатів.

Тестова інфраструктура побудована на основі фреймворку `pytest` і покриває весь ключовий пайплайн застосунку. Тестовий набір налічує 63 тести, що охоплюють усі ключові модулі застосунку та успішно виконуються приблизно за 7 секунд, як показано на рис. 3.6.1.



```
(venv) PS E:\forest_fire_simulator> pytest -q tests/test_analysis_summary.py tests/test_censoring_audit.py tests/test_no_ignition_handling.py
.py tests/test_engine_integration.py tests/test_metrics.py tests/test_run_experiments_cli.py tests/test_scenarios_sensitivity.py tests/test_sp
_wind_model.py
..... [100%]
63 passed in 7.33s
(venv) PS E:\forest_fire_simulator>
```

Рис. 3.6.1 – Результат виконання тестового набору `pytest`

Тести ядра симулятора перевіряють коректність виконання одного кроку симуляції для базових сценаріїв: без дощу, з дощем та з блискавкою. Окремо тестується поведінка вітру, тобто перевіряється, що за однакових стохастичних умов поширення у напрямку вітру не повинно бути слабшим, ніж у сценарії без врахування вітру. Окрема група тестів стосується метрик. Вона перевіряє правильність обчислень типових і граничних випадків, зокрема для порожніх часових рядів, нульових значень, відсутності займання та порожньої сітки. Також перевіряється нормалізація AUC і коректне обмеження значення VAF діапазоном $[0, 1]$. Тести просторових метрик перевіряють маски вигорілих клітин з різною структурою, зокрема порожню маску вигорання, роз'єднані кластери та коректність визначення 8-зв'язних компонент. Окремий набір тестів покриває CLI, завантаження сценарних конфігурацій та генерацію звітів, включно з перевіркою наявності всіх обов'язкових секцій у Markdown і HTML-звіті та коректністю обробки прогонів, які були зупинені через досягнення ліміту кроків.

4. РЕЗУЛЬТАТИ СИМУЛЯЦІЙНИХ ЕКСПЕРИМЕНТІВ

- 4.1. Базові сценарії та верифікація поведінки симулятора
- 4.2. Вплив погодних умов на динаміку пожежі
- 4.3. Вплив типу та щільності рослинності
- 4.4. Аналіз результатів серійних експериментів

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. UNDRR: The invisible cost of wildfire disasters in 2025 [Електронний ресурс]:
Стаття про втрати пов'язані з лісовими пожежами у 2025 році – 13 січня
2026. – Режим доступу:
<https://www.undrr.org/news/invisible-costs-wildfire-disasters-2025>